Sklep z Butami – Bartosz Dziewit, Oliwia Opałacz 3AP

1. Wstęp
1.1. Cel projektu.....5
1.2.Zakres projektu.....5
1.3. Użytkownicy systemu.....5
2. Opis ogólny
2.1. Przeznaczenie witryny.....6
2.2. Główne funkcjonalności.....6
2.3. Wymagania techniczne.....6
2.4. Struktura katalogów i plików projektu.....7
3. Struktura strony internetowej
3.1. Strona główna (index.php).....7
3.2. Sklep (sklep.php).....10
3.3. Koszyk (koszyk.php).....11
3.4. Kontakt (kontakt.php).....12
3.5. Opinie (opinie.php).....14
3.6. Aktualności (aktualnosci.php).....15
3.7. Panel logowania i rejestracji (login.php, register.php).....16
4. Elementy statyczne
4.1. HTML – struktura dokumentów.....18
4.2. CSS – stylizacja (style.css).....21
4.3. Biblioteki zewnętrzne.....22
5. JavaScript
5.1. Slider z aktualnościami.....22
5.2. Obsługa przycisków i efektów interaktywnych.....22
6. Elementy dynamiczne (PHP)

6.1. Obsługa logowania i rejestracji użytkownika.....28
6.2. Pobieranie danych z bazy.....29
6.3. Obsługa formularza kontaktowego.....30
6.4. Panel administracyjny.....32
7. Baza danych
7.1. Projekt struktury bazy danych.....39
7.2. Opis tabel i pól.....40
7.3. Relacje między tabelami.....50
7.4. Przykładowe zapytania SQL.....51
8. Bezpieczeństwo
8.1. Walidacja danych wejściowych.....55
8.2. Obsługa sesji i uwierzytelnianie.....56
9. Możliwości rozbudowy
9.1. Dodanie wyszukiwarki.....57
9.2. System ocen i komentarzy.....57
9.3. System rezerwacji online.....57
9.4. Responsywność i aplikacja mobilna.....57
10. Załączniki
10.1. Fragmenty kodu źródłowego z komentarzami.....58
10.2. Zrzuty ekranu działania systemu.....66
10.3. Diagram bazy danych.....75
10.4. Lista błędów/uwag i pomysłów na ulepszenia.....76

1.Wstęp

1.1. Cel projektu

Głównym celem projektu jest stworzenie responsywnej i funkcjonalnej strony internetowej sklepu z butami, która umożliwi użytkownikom przeglądanie ofert, filtrowanie produktów, dodawanie ich do koszyka oraz składanie zamówień online w sposób szybki i intuicyjny.

1.2. Zakres projektu

1. Strona główna
   - slider
   - sekcje z bestsellerami, zaletami, partnerami
   - krótkie informacje o sklepie
2. Katalog produktów (sklep)
   - podział na rodzaj butów oraz marek
   - Możliwość filtrowania i sortowania (według nazwy, rodzaj, cena, marka)
   - Lista produktów z miniaturkami, ceną
3. Strona produktu
   - Szczegółowe informacje o produkcie
   - Galeria zdjęć
   - Przycisk "Dodaj do koszyka"
4. Koszyk
   - Podsumowanie produktów
   - Finalizacja zamówienia
5. Panel administracyjny
   - Logowanie administratora
   - Dodawanie, edycja i usuwanie produktów
   - Zarządzanie zamówieniami i stanami magazynowymi
6. Inne podstrony
   - Kontakt (formularz, najczęściej zadawane pytania, dane kontaktowe)
   - Opinie (na temat sklepu)
   - Aktualności (nowości związane z produktami)

1.3. Użytkownicy systemu

1. Klient

Osoba odwiedzająca stronę w celu przeglądania i zakupu produktów. Uprawnienia i możliwości:

- Przeglądanie katalogu produktów
- Filtrowanie i sortowanie produktów
- Przeglądanie szczegółów produktu
- Dodawanie produktów do koszyka
- Finalizacja zakupu
- Rejestracja i logowanie

- Kontakt ze sklepem

2. Właściciel sklepu
- Logowanie do panelu administracyjnego
- Dodawanie, edytowanie i usuwanie produktów
- Obsługa zamówień
- Zarządzanie klientami i kontami użytkowników

3. Pracownik
- Podgląd nowych zamówień -
Aktualizacja statusu zamówienia

2. Opis ogólny

2.1. Przeznaczenie witryny

Strona została zaprojektowania z myślą o umożliwieniu użytkownikom wygodnego i intuicyjnego przeglądania, wyszukiwania oraz zakupu obuwia online. Strona pełni funkcję sklepu internetowego oferującego różne rodzaje butów – zarówno dla kobiet, mężczyzn, jak i dzieci. Głównymi celami witryny są:
- prezentacja produktów
- sprzedaż online
- rejestracja i logowanie użytkowników - zarządzanie produktami przez administratora
- promowanie marki.

Witryna została zaprojektowana z naciskiem na użyteczność, estetykę i funkcjonalność, umożliwiając łatwe poruszanie się po kategoriach produktów, szybkie filtrowanie i bezproblemowe składanie zamówień.

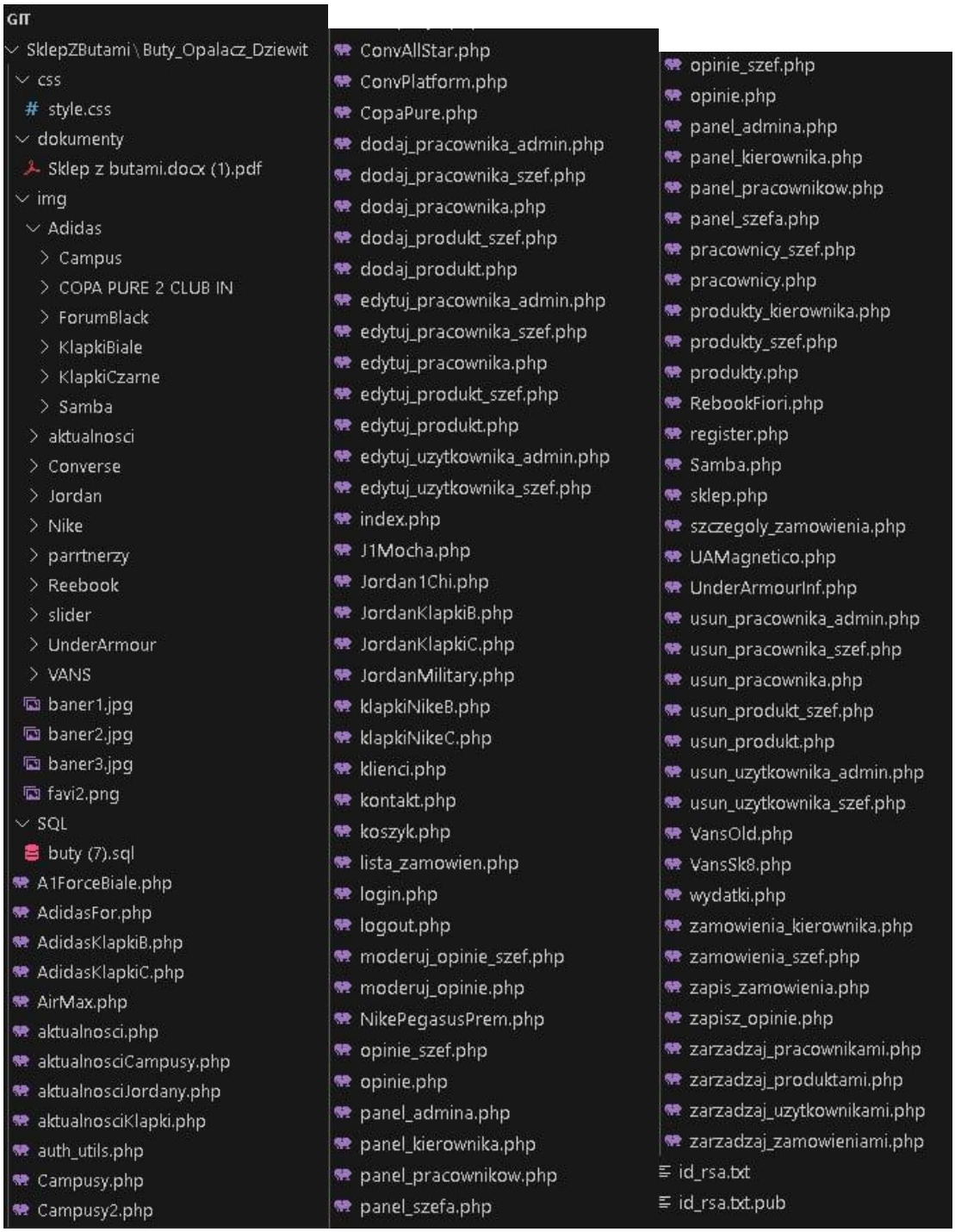
2.2. Główne funkcjonalności

Strona umożliwia rejestracje nowych użytkowników oraz logowanie się już istniejących tj. zwykłego użytkownika, pracownika czy właściela, którzy mają odpowiednie uprawnienia w witrynie. Możliwość dodawania opinii czy wysyłania wiadomości do firmy, dodawanie produktów do koszyka które można usunąć lub kupić. Wyszukiwanie produktów po wybranych filtrach i aktualności które po kliknięciu przenoszą do odpowiednich produktów.

2.3. Wymagania techniczne

Należy uruchomić XAMPP'a oraz zaimportować bazę danych z butami do poprawnego działania witryny. Dowolna przeglądarka, HTML, CSS, JavaScript, PHP.

2.4. Struktura katalogów i plików projektu



3. Struktura strony internetowej

3.1 Strona główna (index.php)

1. Nagłówek

```
    <!-- Nagłówek i menu nawigacyjne -->
    <header>
        <a href="index.php" class="active">Strona Główna</a>
        <a href="sklep.php">Sklep</a>
        <a href="koszyk.php">Koszyk</a>
        <a href="kontakt.php">Kontakt</a>
        <a href="opinie.php">Opinie</a>
        <a href="aktualnosci.php">Aktualności</a>

        <!-- Wyświetlenie statusu logowania -->
<?php if ($zalogowany): ?>
    <span style="float:right; margin-left: 10px; color:#007bff; font-weight: bold;">
        Witaj, <?= htmlspecialchars($_SESSION['username']) ?>! (<?= $rola ?>)
    </span>
    <a href="logout.php" style="float:right;" class="zg">Wyloguj</a>
<?php else: ?>
    <a href="login.php" class="zg">Zaloguj</a>
    <a href="register.php" class="zg">Zarejestruj</a>
<?php endif; ?>
<!-- Linki do paneli dostępne zgodnie z rolą użytkownika -->
<?php if (czy_ma_role(['szef'])): ?>
    <a href="panel_szefa.php">Panel Szefa</a>
<?php endif; ?>

        <?php if (czy_ma_role(['kierownik', 'admin', 'szef'])): ?>
            <a href="panel_kierownika.php">Panel Kierownika</a>
        <?php endif; ?>

        <?php if (czy_ma_role(['Pracownik sklepu', 'kierownik', 'admin', 'szef'])): ?>
            <a href="panel_pracownikow.php">Panel Pracownika</a>
        <?php endif; ?>

    </header>
```

2. Sekcja nawigacji

```
<nav>
<p class="prz">Buty</p>
<div class="zbior">
    <a href="sklep.php?type=Sneakersy">Sneakersy</a>
    <a href="sklep.php?type=Trampki">Trampki</a>
    <a href="sklep.php?type=Biegania">Buty do biegania</a>
    <a href="sklep.php?type=Treningowe">Buty treningowe</a>
    <a href="sklep.php?type=Klapki">Klapki</a>
</div>
</nav>
```

3. Główna zawartość

```
<main>
    <!-- Sekcja slidera promocyjnego -->
    <div class="pokaz">
        <button onclick="plus(-1)">&#10094;</button>
        <!-- Obrazy w sliderze (każdy jako link do aktualności) -->
        <a href="aktualnosciCampusy.php"><img class="pokazslajdow" src="img/slider/elo.jpg" style="width:100%"></a>
        <a href="aktualnosciJordany.php"><img class="pokazslajdow" src="img/slider/jord.jpg" style="width:100%"></a>
        <a href="aktualnosciKlapki.php"><img class="pokazslajdow" src="img/slider/klapki.jpg" style="width:100%"></a>

        <button onclick="plus(1)">&#10095;</button>
    </div>
    <!-- Sekcja bestsellerów -->
    <section class="bestsellery">
        <h2>Nasze Bestsellery</h2>
        <div class="produkty-grid">
            <div class="produkt-card">
                <img src="img/Nike/AF1/AF1white.jpg" alt="Nike Air Force 1" />
                <h3>Nike Air Force 1</h3>
                <p class="cena">499 zł</p>
                <a href="A1ForceBiale.php" class="btn-zobacz">Zobacz</a>
            </div>

            <div class="produkt-card">
                <img src="img/Jordan/Mocha/Mocha1.jpeg" alt="Nike Air Jordan 1 Mocha" />
                <h3>Jordan 1 High Mocha</h3>
                <p class="cena">1249 zł</p>
                <a href="J1Mocha.php" class="btn-zobacz">Zobacz</a>
            </div>

            <div class="produkt-card">
                <img src="img/Adidas/Campus/1.avif" alt="Adidas Campus 00s" />
                <h3>Adidas Campus 00s</h3>
                <p class="cena">529 zł</p>
                <a href="Campusy2.php" class="btn-zobacz">Zobacz</a>
            </div>

            <div class="produkt-card">
                <img src="img/Adidas/ForumBlack/Forum1.jpg" alt="Adidas Forum Low" />
                <h3>Adidas Forum Low</h3>
                <p class="cena">499 zł</p>
                <a href="AdidasFor.php" class="btn-zobacz">Zobacz</a>
            </div>
        </div>
    </section>
```

```
    <!-- Sekcja dlaczego warto wybrać sklep -->
    <section class="why-us">
        <h2>Dlaczego warto wybrać nas?</h2>
        <div class="why-cards">
            <div class="why-card">
                <i class="fas fa-shipping-fast"></i>
                <h3>Darmowa dostawa</h3>
                <p>Przy zamówieniach od 300 zł, przesyłka gratis!</p>
            </div>
            <div class="why-card">
                <i class="fas fa-certificate"></i>
                <h3>Oryginalne produkty</h3>
                <p>100% autentyczne, markowe obuwie od sprawdzonych dostawców.</p>
            </div>
            <div class="why-card">
                <i class="fas fa-undo-alt"></i>
                <h3>14 dni na zwrot</h3>
                <p>Bezproblemowy zwrot towaru do 14 dni od zakupu.</p>
            </div>
        </div>
    </section>

    <!-- Sekcja partnerów -->
    <section class="partners">
        <h2>Nasi Partnerzy</h2>
        <div class="partner-logos">
            <img src="img/parrtnerzy/nike.png" alt="Nike">
            <img src="img/parrtnerzy/adidas.png" alt="Adidas">
            <img src="img/parrtnerzy/reebok.png" alt="reebok">
            <img src="img/parrtnerzy/under.png" alt="under">
        </div>
<footer class="footer">
    <div class="footer-container">
        <div class="footer-column">
            <h3>Kontakt</h3>
            <p>Buty Opalacz Dziewit</p>
            <p>ul. Kwiatowa 30, Mielec </p>
            <p>Tel: <a href="tel:+48123456789">+48 123 456 789</a></p>
            <p>Email: <a href="mailto:kontakt@butyopalacz.pl">kontakt@butyopalacz.pl</a></p>
        </div>
        <div class="footer-column">
            <h3>Godziny otwarcia</h3>
            <p>Poniedziałek – Piątek: 9:00 – 18:00</p>
            <p>Sobota: 10:00 – 14:00</p>
            <p>Niedziela: nieczynne</p>
        </div>
        <div class="footer-column">
            <h3>Śledź nas</h3>
            <div class="social-icons">
                <a href="https://facebook.com/butyopalacz" target="_blank" aria-label="Facebook">
                    <i class="fab fa-facebook-f"></i>
                </a>
                <a href="https://instagram.com/butyopalacz" target="_blank" aria-label="Instagram">
                    <i class="fab fa-instagram"></i>
                </a>
                <a href="https://twitter.com/butyopalacz" target="_blank" aria-label="Twitter">
                    <i class="fab fa-twitter"></i>
                </a>
            </div>
        </div>
    </div>
    <div class="footer-bottom">
        <p>&copy; 2025 Buty Opalacz Dziewit. Wszelkie prawa zastrzeżone.</p>
    </div>
</footer>
```

3.2 Sklep (sklep.php)

1. Nagłówek i stopka strony pozostaje bez zmian 2.

Sekcja nawigacji

```
<nav>
 <div class="prz">Marki</div>
 <div class="zbior brand-filter"> <a href="#" data-brand="">Wszystkie</a>
   <a href="#" data-brand="Nike">Nike</a>
   <a href="#" data-brand="Adidas">Adidas</a>
   <a href="#" data-brand="Jordan">Jordan</a>
   <a href="#" data-brand="Vans">Vans</a>
   <a href="#" data-brand="Under">Under Armour</a>
   <a href="#" data-brand="Converse">Converse</a>
 </div>

 <div class="prz" style="margin-top: 20px;"></div>
 <div class="zbior type-filter"> <a href="#" data-type="Sneakersy">Sneakersy</a>
   <a href="#" data-type="Trampki">Trampki</a>
   <a href="#" data-type="Klapki">Klapki</a>
   <a href="#" data-type="Treningowe">Treningowe</a>
   <a href="#" data-type="Biegania">Do biegania</a>
 </div>
</nav>
```

3. Główna zawartość

- sekcja z filtrami

```
 <select id="filterType">
<option value="">Wszystkie rodzaje</option>
<option value="Sneakersy" <?= $selectedType === 'Sneakersy' ? 'selected' : '' ?>>Sneakersy</option>
<option value="Trampki" <?= $selectedType === 'Trampki' ? 'selected' : '' ?>>Trampki</option>
<option value="Klapki" <?= $selectedType === 'Klapki' ? 'selected' : '' ?>>Klapki</option>
<option value="Treningowe" <?= $selectedType === 'Treningowe' ? 'selected' : '' ?>>Buty Treningowe</option>
<option value="Biegania" <?= $selectedType === 'Biegania' ? 'selected' : '' ?>>Buty do Biegania</option>
</select>

  <label>
    Cena do:
    <input type="range" id="priceRange" min="0" max="2000" step="50" value="2000" />
    <span id="priceValue">2000 zł</span>
  </label>

  <select id="sortOrder">
    <option value="">Sortuj według ceny</option>
    <option value="asc">Cena rosnąco</option>
    <option value="desc">Cena malejąco</option>
  </select>
```

-sekcja z produktami np.

```
<section class="bestsellery">
 <h2>Nasze Produkty</h2>
 <div class="produkty-grid" id="productList">
   <div class="produkt-card" data-brand="Nike" data-price="499" data-type="Sneakersy">
     <img src="img/Nike/AF1/AF1white.jpg" alt="Air Force 1 Białe" />
     <h3>Nike Air Force 1 Białe</h3>
     <p class="cena">499 zł</p>
     <a href="A1ForceBiale.php" class="btn-zobacz">Zobacz</a>
   </div>

   <div class="produkt-card" data-brand="Jordan" data-price="1249" data-type="Sneakersy">
     <img src="img/Jordan/Mocha/Mocha1.jpeg" alt="Jordan 1 Mocha" />
     <h3>Jordan 1 Mocha</h3>
     <p class="cena">1249 zł</p>
     <a href="J1Mocha.php" class="btn-zobacz">Zobacz</a>
   </div>

   <div class="produkt-card" data-brand="Adidas" data-price="529" data-type="Sneakersy">
     <img src="img/Adidas/Campus/1.avif" alt="Adidas Campus 00s" />
     <h3>Adidas Campus 00s Beżowe</h3>
     <p class="cena">529 zł</p>
     <a href="Campusy.php" class="btn-zobacz">Zobacz</a>
   </div>

   <div class="produkt-card" data-brand="Adidas" data-price="529" data-type="Sneakersy">
     <img src="img/Adidas/Campus/campus1.jpg" alt="Adidas Campus 00s Czarne" />
     <h3>Adidas Campus 00s Czarne</h3>
     <p class="cena">529 zł</p>
     <a href="Campusy2.php" class="btn-zobacz">Zobacz</a>
   </div>

   <div class="produkt-card" data-brand="Adidas" data-price="429" data-type="Sneakersy">
     <img src="img/Adidas/Samba/samba1.jpg" alt="Adidas Samba OG White" />
     <h3>Adidas Samba OG</h3>
     <p class="cena">429 zł</p>
     <a href="Samba.php" class="btn-zobacz">Zobacz</a>
   </div>

   <div class="produkt-card" data-brand="Jordan" data-price="1399" data-type="Sneakersy">
     <img src="img/Jordan/Military/Military1.jpg" alt="Jordan 4 Military Black" />
     <h3>Jordan 4 Military Black</h3>
     <p class="cena">1399 zł</p>
     <a href="JordanMilitary.php" class="btn-zobacz">Zobacz</a>
   </div>
```

3.3 Koszyk (koszyk.php)

1. Nagłówek oraz stopka nie ulega zmianom, brak sekcji nawigacji
2. Główna zawartośc

```php
<main>
    <div class="koszyk-container">
        <h1>Twój koszyk</h1>

        <?php if (!empty($koszyk)): ?>
            <form method="post" action="koszyk.php" id="aktualizacja-koszyka-form">
                <?php foreach ($koszyk as $i => $produkt): ?>
                    <div class="produkt-w-koszyku">
                        <div class="produkt-info">
                            <img src="<?= htmlspecialchars($produkt['zdjecie']) ?>"
                                alt="<?= htmlspecialchars($produkt['nazwa']) ?>">
                            <div class="produkt-dane">
                                <span><?= htmlspecialchars($produkt['nazwa']) ?></span>
                                <span>Rozmiar: <?= htmlspecialchars($produkt['rozmiar']) ?></span>
                                <span>Cena: <?= number_format($produkt['cena'], 2) ?> zł</span>
                                <label>
                                    Ilość:
                                    <input type="number" name="ilosc[<?= $i ?>]" value="<?= $produkt['ilosc'] ?>"
                                        min="1" class="ilosc-input">
                                </label>
                            </div>
                        </div>
                        <div>
                            <button type="button" onclick="usunProdukt(<?= $i ?>)" class="usun-btn">Usuń</button>
                        </div>
                    </div>
                <?php endforeach; ?>
                <div class="suma-container">
                    Suma: <?= number_format(oblicz_sume_koszyka(), 2) ?> zł
                    <br>
                    <button type="submit" name="aktualizuj_koszyk" value="1" class="kup-btn">Aktualizuj koszyk</button>
                    <button type="submit" name="kup_teraz" class="kup-btn">Kup Teraz</button>
                </div>
            </form>

        <?php else: ?>
            <p class="pusty">Koszyk jest pusty.</p>
        <?php endif; ?>

    </div>
</main>
```

3.4 Kontakt (kontakt.php)

1. Nagłówek, stopka strony nie ulegają zmian
2. Główna zawartość

```
<main>
    <div class="calosc">
        <div id="wstep">
            <h2>Cześć! Jak możemy pomóc?</h2>
        </div>
        <div class="uwaga">
            <h2>Uwaga!</h2>
            <p>W związku z dużą liczbą zgłoszeń, czas oczekiwania na rozmowę z
            konsultantem na infolinii znacznie się wydłużył.
            Jeśli nie możesz czekać, skontaktuj się z nami pod adresem <b>info@buty.pl</b><br><br>
            Prosimy o wskazanie numeru zamówienia oraz tematu sprawy w tytule wiadomości - pomoże
            nam to szybciej i skuteczniej zająć się Twoim zgłoszeniem.<br><br>
            Dziękujemy za zrozumienie i prosimy o cierpliwość.</p>
        </div><br><br><br>
        <div class="zbior">
<div class="pytanie">Kiedy dostanę zwrot pieniędzy?</div>
<div class="odpowiedz">
  <p><strong>Zwrot w sklepie internetowym</strong><br>
  Pieniądze otrzymasz maksymalnie do 14 dni. Zazwyczaj jest to kilka dni roboczych.</p><br>

  <p><strong>Zwrot produktów opłaconych metodą PayPo</strong><br>
  Pieniądze otrzymasz przez serwis PayPo. Jeśli zwrot produktów nastąpił w ciągu 30 dni
  od zakupu, PayPo zaktualizuje Twoje saldo w ciągu 5 dni roboczych.</p><br>

  <p><strong>Zwrot produktów opłaconych Kartą Podarunkową</strong><br>
  Pieniądze otrzymasz na nową Kartę Podarunkową, którą wyślemy do Ciebie mailem.</p><br>

  <p><strong>Zwrot w sklepie stacjonarnym</strong><br>
  Zwrot środków nastąpi w formie doładowania Karty Podarunkowej BUTYxHalfPrice według zasad
  z Regulaminu Kart Podarunkowych. W razie
  awarii terminy mogą się wydłużyć o kolejne 5 dni roboczych.</p>
</div>
div>
```

```
<div class="zbior">
  <div class="pytanie">Jak mogę złożyć reklamację?</div>
  <div class="odpowiedz">
    <p>Aby złożyć reklamację, skontaktuj się z działem obsługi klienta lub odwiedź najbliższy sklep stacjonarny.</p>
  </div>
</div>

<div class="zbior">
  <div class="pytanie">Co zrobić jeśli przerwało mi płatność?</div>
  <div class="odpowiedz">
    <p>Sprawdź status transakcji w historii płatności. Jeśli płatność
      nie została pobrana, możesz złożyć zamówienie ponownie.</p>
  </div>
</div>

<div class="zbior">
  <div class="pytanie">Jak mogę zwrócić produkt?</div>
  <div class="odpowiedz">
    <p>Nasze zwroty są szybkie i proste:<br>
      - Jeśli zwracasz do 3 produktów włącznie, skorzystaj z InPost Paczkomat® 24/7,<br>
      - Jeśli zwracasz 4 lub więcej produktów, zwrócisz je za pośrednictwem Kuriera DPD.<br><br>
  Każdy produkt możesz też zwrócić w najbliższym sklepie stacjonarnym.<br><br>
  Chcesz zwrócić produkt przez internet?<br>
  1. Zaloguj się i znajdź zakładkę zwroty na Twoim koncie. <br>
  2. Wybierz, które produkty chcesz zwrócić.  <br>
  3. Idź do Paczkomatu® albo czekaj na kuriera.  <br><br>
  Jeśli zwracasz produkty Paczkomatem®, wyślemy do Ciebie SMS z kodem do dowolnego
  Paczkomatu®. Nie musisz drukować etykiety.<br>
  Jeśli zwracasz Kurierem DPD podaj dokładny adres, z którego chcesz
  nadać paczkę. Później zmiana adresu nie będzie możliwa.</p>
  </div>
</div>
```

```
<section class="kontakt">
<h1>Czy masz inne pytania? Napisz do nas</h1>
<form action="" method="post">
<input type="text" name="imie" placeholder="Twoje imię" required>
<input type="email" name="email" placeholder="Twój email" required>
<textarea name="pytanie" placeholder="Napisz tutaj swoje pytanie" required></textarea>
<button type="submit" name="submit" class="submit">Wyślij</button>
</form>
</section>
<div class="info">
    <h2>Informacje kontaktowe</h2>
    <p>Buty.pl - Mielec<br>
    +48 609 350 471<br>
    info@buty.pl</p>
</div>
```

3.5 Opinie (opinie.php)

1. Nagłówek, stopka strony nie ulegają zmian 2.
Główna zawartość

```html
<main class="container opinie">
    <h1>Opinie naszych klientów</h1>

    <section class="lista-opinii" id="lista-opinii">
        <article class="opinia">
            <h3>Anna K.</h3>
            <p>Super obsługa i szybka dostawa. Buty są oryginalne i dobrze zapakowane. Polecam!</p>
            <button class="usun-opinie" hidden>Usuń</button>
        </article>

        <article class="opinia">
            <h3>Mateusz W.</h3>
            <p>Kupiłem Jordany przyszły w 2 dni. Wszystko zgodne z opisem.</p>
            <button class="usun-opinie" hidden>Usuń</button>
        </article>

        <article class="opinia">
            <h3>Karolina M.</h3>
            <p>Duży wybór modeli i dobre ceny. Na pewno wrócę po więcej!</p>
            <button class="usun-opinie" hidden>Usuń</button>
        </article>

    </section>

    <section class="formularz-opinii">
        <h2>Dodaj swoją opinię</h2>
        <form id="formularz-opinii">
            <input type="text" id="imie" placeholder="Twoje imię" required />
            <textarea id="tresc" placeholder="Twoja opinia..." required></textarea>
            <button type="submit">Wyślij</button>
        </form>
    </section>
</main>
```

3.6 Aktualności (aktualnosci.php)

1. Nagłówek, stopka strony nie ulegają zmian
2. Główna zawartość

```
    <main class="news-section">
        <h1 class="news-title">📰 Najnowsze aktualności</h1>
        <div class="news-cards">
            <a href="aktualnosciCampusy.php" style="text-decoration: none; color: inherit; display: block;">
    <div class="news-card">
        <img src="img/aktualnosci/2.png" class="nowosc" alt="Przykładowe zdjęcie">
        <div class="news-card-content">
            <h3>👟 Nowa Kolekcja Campusów</h3>
            <p>Przedstawiamy najnowszą kolekcję butów Adidas Campus – klasyka
            w nowoczesnym wydaniu! Idealne na co dzień, wykonane z wysokiej jakości materiałów,
            zapewniają maksymalny komfort i styl. Dostępne w wielu wariantach kolorystycznych.</p>
            <p class="date">2 maja 2025</p>
        </div>
    </a>
        </div>

<br>
<a href="aktualnosciJordany.php" style="text-decoration: none; color: inherit; display: block;">
    <div class="news-card">
        <img src="img/aktualnosci/3.png" class="nowosc" alt="Buty Jordan">
        <div class="news-card-content">
            <h3>🔥 Nowa kolekcja Jordan & Nike</h3>
            <p>Do oferty trafiły limitowane modele Air Jordan 1, Air Jordan 1 High oraz nowości
            od Nike! Wyjątkowy design, precyzja wykonania i wygoda – to cechy, które wyróżniają
            tę kolekcję. Idealne dla fanów streetwearu i sneakerheadów.</p>
            <p class="date"> 28 kwietnia 2025</p>
        </div>
    </div>
</a>

<br>
<a href="aktualnosciKlapki.php" style="text-decoration: none; color: inherit; display: block;">
    <div class="news-card">
        <img src="img/aktualnosci/5.png" alt="klapki">
        <div class="news-card-content">
            <h3>👟 Nowa kolekcja klapkow</h3>
            <p>Nowa kolekcja Klapków! Obczaj co nowego czeka na ciebie! Klapki z różnych
            marek - NIKE, ADIDAS i JORDAN w bardzo korzystnych cenach zobaczcie sami!!!</p>
            <span class="date"> 29 kwietnia 2025</span>
        </div>
    </div>
    </a>
    </main>
```

3.7. Panel logowania i rejestracji (login.php, register.php)

1. Logowanie

1. Nagłówek, stopka strony nie ulegają zmian 2. Główna zawartość

```
<div class="auth-container">
    <h2>Zaloguj się</h2>
    <?php if (!empty($wiadomosc_bledu)): ?>
        <p class="error-message"><?= $wiadomosc_bledu ?></p>
    <?php endif; ?>
    <form method="post">
        <input type="text" name="username" placeholder="Nazwa użytkownika" required />
        <input type="password" name="password" placeholder="Hasło" required />
        <button type="submit">Zaloguj</button>
    </form>
    <div class="register-link">
        <p>Nie masz konta?</p>
        <a href="register.php">Zarejestruj się</a>
    </div>
</div>
```

2. Rejestracja
1. Nagłówek, stopka strony nie ulegają zmian 2.
Główna zawartość

```
<div class="auth-container">
    <h2>Załóż konto</h2>
    <form action="register.php" method="POST">
        <input type="text" name="username" placeholder="Nazwa użytkownika" required>
        <input type="email" name="email" placeholder="Email" required>
        <input type="password" name="password" placeholder="Hasło" required>

        <button type="submit">Zarejestruj się</button>
    </form>
</div>
```

4. Elementy statyczne

4.1. HTML – struktura dokumentów

Struktura dokumentów HTML w projekcie została oparta o semantyczny standard HTML5. Poszczególne podstrony tworzą logicznie zorganizowaną architekturę, umożliwiającą zarówno łatwą nawigację użytkownika, jak i dalszy rozwój witryny.

Główne komponenty struktury:

- **`<!DOCTYPE html>`** – deklaracja typu dokumentu HTML5.
- **`<html lang="pl">`** – ustawienie języka na polski.
- **`<head>`** – zawiera metadane, tytuł strony, linki do arkuszy stylów (`style.css`), czcionek (Google Fonts) oraz bibliotek zewnętrznych (np. Font Awesome, Swiper.js).
- **`<body>`** – zawiera widoczną dla użytkownika strukturę strony.

Układ treści strony (<body>):

- **`<header>`** – nagłówek strony:
    - logo lub nazwa serwisu (opcjonalnie), ○ główne menu nawigacyjne (linki do podstron), ○ przyciski logowania i rejestracji (z klasą `.zg` – odpowiednio stylizowane).
- **`<nav>`** – pasek nawigacyjny z kategoriami produktów:
    - lista linków do podstron z produktami (`Sneakersy`, `Trampki`, `Klapki` itd.), ○ zastosowano elastyczny układ (`flex`) oraz stylizowane przyciski.
- **`<main>`** – główna zawartość strony:

- slider z aktualnościami (zdjęcia i przyciski sterujące), ○ sekcja bestsellerów – dynamiczne generowanie kart produktów (`.produkt-card`), ○ dział "Dlaczego my?" – marketingowe informacje w kartach z ikonami, ○ lista partnerów – logo znanych marek.

- **`<footer>`** – stopka strony: ○ dane kontaktowe, linki informacyjne, odnośniki do mediów społecznościowych, ○ podpis i prawa autorskie, ○ zastosowano responsywny podział na kolumny.

Dodatkowe podstrony:

Projekt zawiera szereg plików `.php` reprezentujących różne widoki i funkcje, m.in.:

- `index.php` – strona główna, `produkty.php`, `sklep.php` – katalogi
- produktów, `login.php`, `register.php` – autoryzacja użytkowników,
- `kontakt.php` – formularz kontaktowy, `panel_admina.php`,
- `panel_pracownikow.php` – panele zarządzające.
-

Uwagi:

- Struktura jest przystosowana do integracji z backendem PHP i bazą danych MySQL.
- Dynamiczne elementy (np. lista produktów, slider, formularze) są osadzone w strukturze HTML przy pomocy PHP (`echo`, `foreach`, `include`).
- Stylizacja i interaktywność są realizowane za pomocą `style.css` oraz skryptów JavaScript.

Przykladowa strona

```php
<!DOCTYPE html>
<html lang="pl">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Sklep z Butami - Strona główna</title>
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.0.0-beta3/css/all.min.css">
    <link rel="stylesheet" href="css/style.css">
    <link rel="icon" href="img/favi2.png" type="image/png">
</head>

<body>
    <div class="wrapper">
        <header>
            <a href="index.php" class="active">Strona Główna</a>
            <a href="sklep.php">Sklep</a>
            <a href="koszyk.php">Koszyk</a>
            <a href="kontakt.php">Kontakt</a>
            <a href="opinie.php">Opinie</a>
            <a href="aktualnosci.php">Aktualności</a>

            <?php if ($zalogowany): ?>
                <span style="float:right; margin-left: 10px; color:#007bff; font-weight: bold;">
                    Witaj, <?= htmlspecialchars($_SESSION['username']) ?>! (<?= $rola ?>)
                </span>
                <a href="logout.php" style="float:right;" class="zg">Wyloguj</a>
            <?php else: ?>
                <a href="login.php" class="zg">Zaloguj</a>
                <a href="register.php" class="zg">Zarejestruj</a>
            <?php endif; ?>

            <?php if (czy_ma_role(['szef'])): ?>
                <a href="panel_szefa.php">Panel Szefa</a>
            <?php endif; ?>

            <?php if (czy_ma_role(['admin', 'szef'])): ?>
                <a href="panel_admina.php">Panel Admina</a>
            <?php endif; ?>

            <?php if (czy_ma_role(['kierownik', 'admin', 'szef'])): ?>
                <a href="panel_kierownika.php">Panel Kierownika</a>
            <?php endif; ?>

            <?php if (czy_ma_role(['Pracownik sklepu', 'kierownik', 'admin', 'szef'])): ?>
                <a href="panel_pracownikow.php">Panel Pracownika</a>
            <?php endif; ?>

        </header>
```

```
<nav>
    <p class="prz">Buty</p>
    <div class="zbior">
        <a href="sklep.php?type=Sneakersy">Sneakersy</a>
        <a href="sklep.php?type=Trampki">Trampki</a>
        <a href="sklep.php?type=Biegania">Buty do biegania</a>
        <a href="sklep.php?type=Treningowe">Buty treningowe</a>
        <a href="sklep.php?type=Klapki">Klapki</a>
    </div>
</nav>

<main>
    <div class="pokaz">
        <button onclick="plus(-1)">&#10094;</button>

        <a href="aktualnosciCampusy.php"><img class="pokazslajdow" src="img/slider/elo.jpg" style="width:100%"></a>
        <a href="aktualnosciJordany.php"><img class="pokazslajdow" src="img/slider/jord.jpg" style="width:100%"></a>
        <a href="aktualnosciKlapki.php"><img class="pokazslajdow" src="img/slider/klapki.jpg" style="width:100%"></a>

        <button onclick="plus(1)">&#10095;</button>
    </div>

    <section class="bestsellery">
        <h2>Nasze Bestsellery</h2>
        <div class="produkty-grid">
            <div class="produkt-card">
                <img src="img/Nike/AF1/AF1white.jpg" alt="Nike Air Force 1" />
                <h3>Nike Air Force 1</h3>
                <p class="cena">499 zł</p>
                <a href="A1ForceBiale.php" class="btn-zobacz">Zobacz</a>
            </div>

            <div class="produkt-card">
                <img src="img/Jordan/Mocha/Mocha1.jpeg" alt="Nike Air Jordan 1 Mocha" />
                <h3>Jordan 1 High Mocha</h3>
                <p class="cena">1249 zł</p>
                <a href="J1Mocha.php" class="btn-zobacz">Zobacz</a>
            </div>

            <div class="produkt-card">
                <img src="img/Adidas/Campus/1.avif" alt="Adidas Campus 00s" />
                <h3>Adidas Campus 00s</h3>
                <p class="cena">529 zł</p>
                <a href="Campusy2.php" class="btn-zobacz">Zobacz</a>
            </div>

            <div class="produkt-card">
                <img src="img/Adidas/ForumBlack/Forum1.jpg" alt="Adidas Forum Low" />
                <h3>Adidas Forum Low</h3>
                <p class="cena">499 zł</p>
                <a href="AdidasFor.php" class="btn-zobacz">Zobacz</a>
            </div>
        </div>
    </section>

    <section class="why-us">
        <h2>Dlaczego warto wybrać nas?</h2>
        <div class="why-cards">
            <div class="why-card">
                <i class="fas fa-shipping-fast"></i>
                <h3>Darmowa dostawa</h3>
                <p>Przy zamówieniach od 300 zł, przesyłka gratis!</p>
            </div>
            <div class="why-card">
                <i class="fas fa-certificate"></i>
                <h3>Oryginalne produkty</h3>
                <p>100% autentyczne, markowe obuwie od sprawdzonych dostawców.</p>
            </div>
            <div class="why-card">
                <i class="fas fa-undo-alt"></i>
                <h3>14 dni na zwrot</h3>
                <p>Bezproblemowy zwrot towaru do 14 dni od zakupu.</p>
            </div>
        </div>
    </section>

    <section class="partners">
        <h2>Nasi Partnerzy</h2>
        <div class="partner-logos">
            <img src="img/parrtnerzy/nike.png" alt="Nike">
            <img src="img/parrtnerzy/adidas.png" alt="Adidas">
            <img src="img/parrtnerzy/reebok.png" alt="reebok">
            <img src="img/parrtnerzy/under.png" alt="under">
        </div>
    </section>
</main>
```

4.2. CSS – stylizacja (`style.css`)

Stylizacja interfejsu została zrealizowana przy użyciu zewnętrznego arkusza `style.css`, znajdującego się w folderze `css`. Styl odpowiada za nowoczesny i przejrzysty wygląd serwisu, z zachowaniem spójnej kolorystyki oraz funkcjonalności wspierającej użytkowników na komputerach i urządzeniach mobilnych.

Główne cechy stylizacji:

- **Reset stylów**: zastosowano `box-sizing: border-box`, zerowanie marginesów i paddingów dla wszystkich elementów (`* {}`), co zapewnia spójność wyświetlania we wszystkich przeglądarkach.
- **Typografia i tło**: przyjemna dla oka czcionka `'Segoe UI'`, kontrastowa kolorystyka z dominującym jasnym tłem (`#f8f9fa`) oraz tekstem w odcieniu ciemnej szarości (`#333`).

Kluczowe komponenty i ich stylizacja:

- **Nagłówek (`header`)** – ciemne tło, przyciski logowania i rejestracji z zaokrąglonymi rogami, efekty `:hover` i `active`.
- **Menu nawigacyjne (`nav`)** – jasne tło z pogrubioną nazwą sekcji oraz elastycznym układem linków (`display: flex`, `flex-wrap: wrap`), które zmieniają kolor i tło przy najechaniu kursorem.
- **Slider (`.pokaz`)** – prosta karuzela zdjęć z przyciskami do przewijania. Stylizacja przycisków z przezroczystym tłem, zaokrąglonymi rogami i efektami przejścia (`transition`).
- **Stopka (`.footer`)** – podzielona na kolumny z linkami, kontaktami oraz ikonami społecznościowymi, z efektami `:hover` i jasnymi akcentami na ciemnym tle.
- **Sekcja bestsellerów (`.bestsellery`)** – siatka produktów oparta na `display: grid`, karty produktów z cieniami i efektami `hover`, stylizowane przyciski zakupu (`.btnzobacz`).
- **Dlaczego my (`.why-us`)** – sekcja marketingowa z ikonami, kartami i efektami unoszenia (`transform: translateY(-5px)`).
- **Sekcja partnerów (`.partners`)** – galeria logo partnerów z efektem `grayscale()` i interaktywnym `hover`.
- **Strona produktu (`.product-page`)** – elastyczny układ treści i zdjęć, miniatury, rozwijane selektory rozmiarów, przyciski „dodaj do koszyka” i „kup teraz”.
- **Opinie (`.opinie`)** – blok z cytowanymi opiniami użytkowników (`<blockquote>`, `.opinia`), zastosowano kursywę, kolory wyróżniające imiona.
- **System filtrów (`.filtry-container`)** – pasek filtrów z polami `input` i `select`, dostosowany szerokością do układu siatki produktów.

- **Kategorie i podkategorie** – rozwijane menu (`.category-group`, `.subcategories`) z ukrytymi pozycjami, które pokazują się po najechaniu, tworząc funkcjonalne menu typu dropdown.

Responsywność:

- Użyto jednostek względnych (`rem`, `%`, `vw`) oraz `flexbox` i `grid`, co pozwala łatwo dostosować układ do różnych rozdzielczości ekranu.
- W kodzie przygotowano strukturalne podstawy pod pełną responsywność (łatwa do wdrożenia przez `@media` queries).

4.3. Biblioteki zewnętrzne

```
<!-- Import ikon Font Awesome -->
<link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.0.0-beta3/css/all.min.css">
```

5. JavaScript

5.1. Slider z aktualnościami

```
<!-- Skrypt do obsługi slidera -->
<script>
    var indeks = 1;
    pok(indeks);
    // Zmiana slajdu o n pozycji
    function plus(n) {
        pok(indeks += n); // Zwiększenie/zmniejszenie indeksu i wyświetlenie slajdu.
    }
    // Wyświetlanie odpowiedniego slajdu
    function pok(n) {
        var i;
        var x = document.getElementsByClassName("pokazslajdow"); // Pobranie wszystkich elementów z klasą "pokazslajdow".
        // Jeśli indeks jest większy niż liczba slajdów, wróć do pierwszego.
        if (n > x.length) {
            indeks = 1
        }
        // Jeśli indeks jest mniejszy niż 1, przejdź do ostatniego slajdu.
        if (n < 1) {
            indeks = x.length
        }
        // Ukrycie wszystkich slajdów.
        for (i = 0; i < x.length; i++) {
            x[i].style.display = "none";
        }
        // Wyświetlenie slajdu o aktualnym indeksie.
        x[indeks - 1].style.display = "block";
    }
</script>
```

5.2. Obsługa przycisków i efektów interaktywnych

1. Obsługa filtrów (sklep.php)

```
// Pobieranie elementów
const poleWyszukiwania = document.getElementById("searchInput");
const filtrMarka = document.getElementById("filterBrand");
const filtrRodzaj = document.getElementById("filterType");
const suwakCeny = document.getElementById("priceRange");
const wartoscCeny = document.getElementById("priceValue");
const sortowanieCeny = document.getElementById("sortOrder");
const listaProduktow = document.getElementById("productList");
const wszystkieProdukty = Array.from(listaProduktow.querySelectorAll(".produkt-card")); // Pobranie wszystkich produktów
const brandLinks = document.querySelectorAll('.brand-filter a'); // Pobranie linków do filtrowania po marce
const typeLinks = document.querySelectorAll('.type-filter a');   // Pobranie linków do filtrowania po typie

// Aktualizacja tekstu przy suwaku ceny
suwakCeny.addEventListener("input", () => {
  wartoscCeny.textContent = suwakCeny.value + " zł"; // Wyświetlanie aktualnej wartości suwaka
  filtrujProdukty(); // Wywołanie funkcji filtrującej po zmianie wartości suwaka
});

// Nasłuchiwanie zmian na filtrach (wyszukiwanie, marka, rodzaj, sortowanie)
[poleWyszukiwania, filtrMarka, filtrRodzaj, sortowanieCeny].forEach(element => {
  element.addEventListener("change", filtrujProdukty); // Dla każdego filtra dodajemy listener na zdarzenie 'change'
});

// Obsługa kliknięcia w markę (filtry w menu bocznym)
brandLinks.forEach(link => {
  link.addEventListener('click', (e) => {
    e.preventDefault(); // Zapobieganie przeładowaniu strony
    const brand = link.dataset.brand; // Pobranie wartości atrybutu data-brand
    filtrMarka.value = brand;         // Ustawienie wartości selecta z marką
    filtrRodzaj.value = '';           // Reset filtra rodzaju
    resetujPozostaleFiltry();        // Reset pozostałych filtrów
    filtrujProdukty();              // Wywołanie funkcji filtrującej
    aktualizujAktywneLinki();       // Aktualizacja wyglądu aktywnych linków
// Obsługa kliknięcia w rodzaj buta (filtry w menu bocznym)
typeLinks.forEach(link => {
  link.addEventListener('click', (e) => {
    e.preventDefault(); // Zapobieganie przeładowaniu strony
    const type = link.dataset.type; // Pobranie wartości atrybutu data-type
    filtrRodzaj.value = type;         // Ustawienie wartości selecta z rodzajem
    filtrMarka.value = '';           // Reset filtra marki
    resetujPozostaleFiltry();        // Reset pozostałych filtrów
    filtrujProdukty();              // Wywołanie funkcji filtrującej
    aktualizujAktywneLinki();       // Aktualizacja wyglądu aktywnych linków
  });
});

// Funkcja resetująca pozostałe filtry (wyszukiwanie, cena, sortowanie)
function resetujPozostaleFiltry() {
  poleWyszukiwania.value = '';       // Reset pola wyszukiwania
  suwakCeny.value = '2000';         // Reset suwaka ceny
  wartoscCeny.textContent = '2000 zł'; // Reset tekstu wyświetlanego przy suwaku
  sortowanieCeny.value = '';         // Reset sortowania
}

// Funkcja aktualizująca wygląd aktywnych linków w menu bocznym
function aktualizujAktywneLinki() {
  // Resetuj wszystkie linki (usuwa klasę active-category)
  brandLinks.forEach(link => link.classList.remove('active-category'));
  typeLinks.forEach(link => link.classList.remove('active-category'));

  // Ustaw aktywne linki na podstawie wybranych filtrów
  const wybranaMarka = filtrMarka.value;
  const wybranyRodzaj = filtrRodzaj.value;

  // Aktywuj link marki, jeśli marka jest wybrana
  if (wybranaMarka) {
    document.querySelector(`.brand-filter a[data-brand="${wybranaMarka}"]`).classList.add('active-category');
  } else {
    document.querySelector('.brand-filter a[data-brand=""]').classList.add('active-category'); // Aktywny link "Wszystkie"
  }

  // Aktywuj link rodzaju buta, jeśli rodzaj jest wybrany
  if (wybranyRodzaj) {
    document.querySelector(`.type-filter a[data-type="${wybranyRodzaj}"]`).classList.add('active-category');
  } else {
    document.querySelector('.type-filter a[data-type=""]').classList.add('active-category'); // Aktywny link "Wszystkie"
  }
}
```

```
// Główna funkcja filtrująca produkty
function filtrujProdukty() {
  const tekstWyszukiwania = poleWyszukiwania.value.toLowerCase(); // Pobranie tekstu z pola wyszukiwania i zmiana na małe litery
  const wybranaMarka = filtrMarka.value;                    // Pobranie wybranej marki
  const wybranyRodzaj = filtrRodzaj.value;                  // Pobranie wybranego rodzaju buta
  const maksCena = parseInt(suwakCeny.value);                // Pobranie maksymalnej ceny jako liczba
  const kolejnoscSortowania = sortowanieCeny.value;          // Pobranie kolejności sortowania

  // Filtrowanie produktów na podstawie wprowadzonych kryteriów
  let pasujaceProdukty = wszystkieProdukty.filter(produkt => {
    const nazwa = produkt.querySelector("h3").textContent.toLowerCase(); // Pobranie nazwy produktu i zmiana na małe litery
    const marka = produkt.dataset.brand;             // Pobranie marki produktu z atrybutu data-brand
    const rodzaj = produkt.dataset.type;           // Pobranie rodzaju produktu z atrybutu data-type
    const cena = parseInt(produkt.dataset.price);     // Pobranie ceny produktu z atrybutu data-price

    // Sprawdzenie, czy produkt spełnia wszystkie kryteria
    return (
      nazwa.includes(tekstWyszukiwania) &&       // Czy nazwa produktu zawiera szukany tekst
      (!wybranaMarka || marka === wybranaMarka) && // Czy marka produktu pasuje do wybranej marki (lub brak wybranej marki)
      (!wybranyRodzaj || rodzaj === wybranyRodzaj) && // Czy rodzaj produktu pasuje do wybranego rodzaju (lub brak wybranego rodzaju)
      cena <= maksCena                           // Czy cena produktu jest mniejsza lub równa maksymalnej cenie
    );
  });

  // Sortowanie produktów
  if (kolejnoscSortowania === "asc") {
    pasujaceProdukty.sort((a, b) => a.dataset.price - b.dataset.price); // Sortowanie rosnąco
  } else if (kolejnoscSortowania === "desc") {
    pasujaceProdukty.sort((a, b) => b.dataset.price - a.dataset.price); // Sortowanie malejąco
  }

  // Aktualizacja widoku listy produktów
  listaProduktow.innerHTML = ""; // Wyczyść aktualną listę produktów
  pasujaceProdukty.forEach(produkt => listaProduktow.appendChild(produkt)); // Dodaj przefiltrowane produkty do listy

  // Aktualizacja wyglądu aktywnych linków w menu bocznym
  aktualizujAktywneLinki();
}
```

```
// Inicjalizacja strony - wykonuje się po załadowaniu DOM
document.addEventListener('DOMContentLoaded', function() {
  // Sprawdź parametry URL
  const urlParams = new URLSearchParams(window.location.search);
  const selectedType = urlParams.get('type');
  const selectedBrand = urlParams.get('brand');

  // Ustaw filtry na podstawie parametrów URL, jeśli istnieją
  if (selectedType) filtrRodzaj.value = selectedType;
  if (selectedBrand) filtrMarka.value = selectedBrand;

  // Wywołaj funkcję filtrującą, aby wyświetlić produkty na starcie
  filtrujProdukty();

  // Przewiń do sekcji produktów, jeśli przekazano parametry w URL
  if (selectedType || selectedBrand) {
    document.querySelector('.bestsellery').scrollIntoView({
      behavior: 'smooth' // Płynne przewijanie
    });
  }
});
```

2. Usuwanie produktu

```
<script>
function usunProdukt(indeks) {
    if (confirm('Czy na pewno chcesz usunąć ten produkt z koszyka?')) {
        window.location.href = 'koszyk.php?usun=' + indeks;
    }
}
</script>
```

3. Skrypt po którym kliknięcie w klase pytanie, wyświetli odpowiedź

```
<script>
  document.querySelectorAll('.pytanie').forEach(item => {
    item.addEventListener('click', () => {
      item.classList.toggle('active');
      const answer = item.nextElementSibling;
      answer.classList.toggle('show');
    });
  });
</script><br><br>
```

4.Opinie

```
// Obsługa asynchronicznego wysyłania formularza opinii za pomocą Fetch API
document.getElementById('formularz-opinii').addEventListener('submit', async function(e) {
    e.preventDefault(); // Zapobiega domyślnej akcji formularza, czyli przeładowaniu strony.

    const imie = document.getElementById('imie').value; // Pobranie wartości z pola imienia.
    const tresc = document.getElementById('tresc').value; // Pobranie wartości z pola treści opinii.

    // Wykonanie żądania POST do serwera (do tego samego pliku 'opinie.php') w celu dodania opinii.
    const res = await fetch('opinie.php', {
        method: 'POST', // Metoda HTTP POST, używana do wysyłania danych do serwera.
        headers: {'Content-Type': 'application/x-www-form-urlencoded'}, // Ustawienie nagłówka informującego serwer o formacie danych
        body: `imie=${encodeURIComponent(imie)}&tresc=${encodeURIComponent(tresc)}` // Dane formularza zakodowane w formacie URL.
    });

    // Sprawdzenie, czy odpowiedź z serwera jest pomyślna (status HTTP 2xx).
    if (res.ok) {
        document.getElementById('imie').value = ''; // Wyczyszczenie pola imienia po pomyślnym wysłaniu.
        document.getElementById('tresc').value = ''; // Wyczyszczenie pola treści opinii.
        wczytajOpinie(); // Wywołanie funkcji wczytującej aktualną listę opinii z serwera.
    }
});
```

```
// Asynchroniczna funkcja do pobierania i wyświetlania opinii z serwera.
async function wczytajOpinie() {
    const res = await fetch('opinie.php'); // Wykonanie żądania GET do serwera (do tego samego pliku) w celu pobrania opinii.
    const data = await res.json(); // Parsowanie odpowiedzi z serwera jako JSON.

    const container = document.getElementById('lista-opinii'); // Pobranie elementu kontenera, w którym będą wyświetlane opinie.
    container.innerHTML = ''; // Wyczyszczenie dotychczasowej zawartości kontenera opinii.

    // Iteracja po każdej opinii pobranej z serwera.
    data.forEach(opinia => {
        const div = document.createElement('div'); // Utworzenie nowego elementu div dla każdej opinii.
        div.classList.add('opinia'); // Dodanie klasy CSS 'opinia' do nowego elementu.
        div.innerHTML = `<strong>${opinia.imie}</strong>: ${opinia.komentarz} <em>(${opinia.data_opinii})</em>`; // Wstawienie danych opinii
        container.appendChild(div); // Dodanie utworzonego elementu opinii do kontenera na stronie.
    });
}
// Wywołanie funkcji wczytującej opinie przy pierwszym załadowaniu strony, aby wyświetlić istniejące opinie.
wczytajOpinie();
```

```
// Po załadowaniu strony – wczytaj opinie z localStorage
document.addEventListener("DOMContentLoaded", function () {
  const zapisane = JSON.parse(localStorage.getItem("opinie")) || [];
  const kontener = document.getElementById("lista-opinii");

  // Dodaj dynamiczne opinie
  zapisane.forEach((opinia, index) => {
    const el = stworzOpiniaElement(opinia.imie, opinia.tresc, index);
    kontener.appendChild(el);
  });

  // Dodaj działanie przyciskom "Usuń" dla opinii statycznych
  document.querySelectorAll(".opinia .usun-opinie").forEach(button => {
    button.addEventListener("click", function () {
      this.parentElement.remove();
    });
  });
});

// Obsługa formularza dodawania opinii
document.getElementById("formularz-opinii").addEventListener("submit", function (e) {
  e.preventDefault();

  const imie = document.getElementById("imie").value.trim();
  const tresc = document.getElementById("tresc").value.trim();

  if (imie && tresc) {
    const nowaOpinia = { imie, tresc };
    const stare = JSON.parse(localStorage.getItem("opinie")) || [];
    stare.push(nowaOpinia);
    localStorage.setItem("opinie", JSON.stringify(stare));

    const el = stworzOpiniaElement(imie, tresc, stare.length - 1);
    document.getElementById("lista-opinii").appendChild(el);
// Funkcja tworzy element opinii z przyciskiem usuń
function stworzOpiniaElement(imie, tresc, index) {
  const el = document.createElement("article");
  el.classList.add("opinia");

  el.innerHTML = `
    <h3>${imie}</h3>
    <p>${tresc}</p>
    <button class="usun-opinie" hidden>Usuń</button>
  `;

  // Obsługa kliknięcia przycisku usuń
  el.querySelector(".usun-opinie").addEventListener("click", function () {
    el.remove();

    // Usunięcie z localStorage
    const opinie = JSON.parse(localStorage.getItem("opinie")) || [];
    opinie.splice(index, 1); // usuń z listy po indeksie
    localStorage.setItem("opinie", JSON.stringify(opinie));
  });

  return el;
}
```

6. Elementy dynamiczne (PHP)

6.1. Obsługa logowania i rejestracji użytkownika

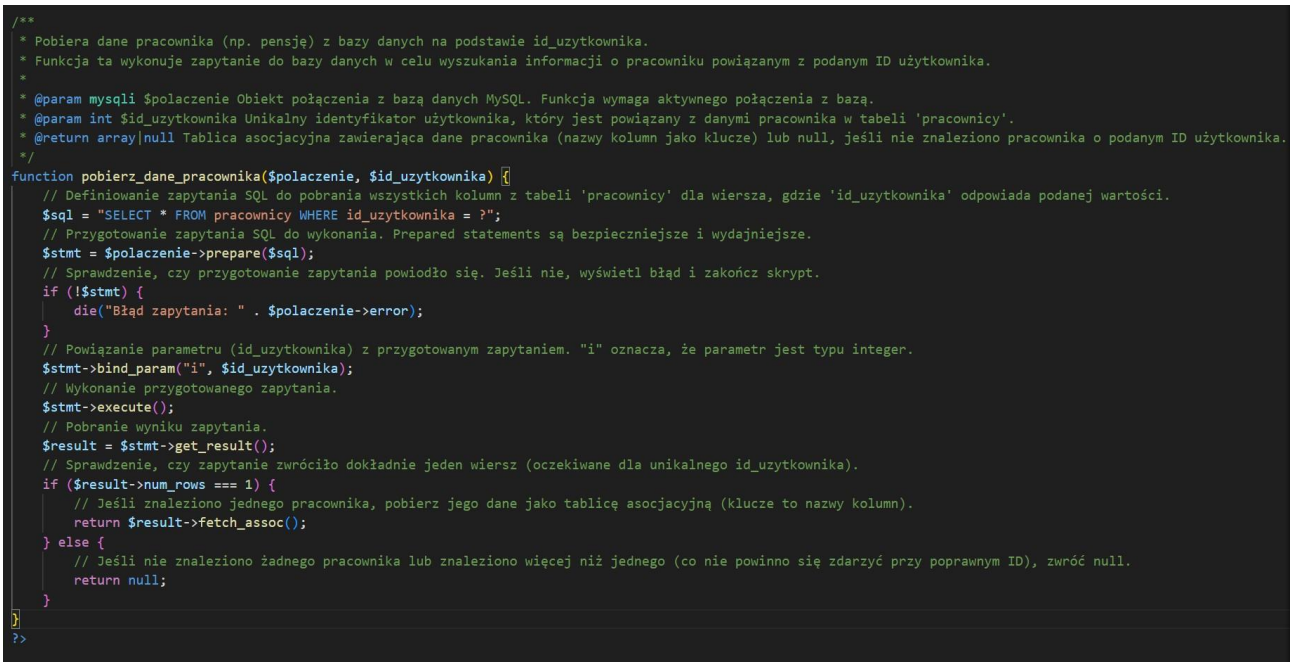
Logowanie i rejestracja klientów odbywa się poprzez formularze, które odwołują się do tabeli `klienci`. W bazie tej znajdują się pola `email`, `haslo` (tekstowe, bez hash) istnieje też zarządzanie pracownikami (`pracownicy`)

```php
<?php
// auth_utils.php

/**
 * Sprawdza, czy użytkownik ma jedną z wymaganych ról.
 * Funkcja ta pobiera rolę aktualnie zalogowanego użytkownika z sesji
 * i porównuje ją z listą dozwolonych ról.
 *
 * @param string|array $wymagane_role Rola (string) lub tablica ról (array), które są dozwolone.
 * Jeśli podano string, sprawdzana jest dokładna zgodność.
 * Jeśli podano tablicę, sprawdzane jest, czy rola użytkownika znajduje się w tej tablicy.
 * @return bool True, jeśli użytkownik ma uprawnienia (jego rola pasuje do jednej z wymaganych),
 * false w przeciwnym razie (użytkownik niezalogowany lub jego rola nie pasuje).
 */
function czy_ma_role($wymagane_role) {
    // Sprawdzenie, czy w sesji istnieje zmienna 'rola'. Jeśli nie, użytkownik nie jest zalogowany i nie ma przypisanej roli.
    if (!isset($_SESSION['rola'])) {
        return false; // Użytkownik niezalogowany
    }

    // Sprawdzenie, czy $wymagane_role jest tablicą.
    if (is_array($wymagane_role)) {
        // Jeśli jest tablicą, sprawdzamy, czy rola użytkownika (pobrana z sesji) znajduje się w tej tablicy za pomocą funkcji in_array().
        return in_array($_SESSION['rola'], $wymagane_role);
    } else {
        // Jeśli $wymagane_role nie jest tablicą (czyli jest stringiem), sprawdzamy, czy rola użytkownika dokładnie odpowiada tej wymaganej roli.
        return $_SESSION['rola'] == $wymagane_role;
    }
}
```

```php
/**
 * Przekierowuje użytkownika i wyświetla komunikat o błędzie, jeśli nie ma uprawnień.
 * Ta funkcja łączy sprawdzenie uprawnień z akcją przekierowania i ustawieniem komunikatu błędu w sesji.
 *
 * @param string|array $wymagane_role Rola (string) lub tablica ról (array), które są dozwolone (tak jak w funkcji czy_ma_role()).
 * @param string $przekierowanie URL, na który użytkownik zostanie przekierowany, jeśli nie ma wymaganych uprawnień.
 * @param string $wiadomosc Komunikat tekstowy, który zostanie zapisany w sesji pod kluczem 'blad_uprawnien' i może być wyświetlony na stronie przekierowania.
 */
function sprawdz_i_przekieruj($wymagane_role, $przekierowanie, $wiadomosc) {
    // Wywołanie funkcji czy_ma_role() w celu sprawdzenia, czy użytkownik posiada wymagane uprawnienia.
    if (!czy_ma_role($wymagane_role)) {
        // Jeśli użytkownik nie ma uprawnień, zapisujemy komunikat o błędzie w zmiennej sesyjnej 'blad_uprawnien'.
        $_SESSION['blad_uprawnien'] = $wiadomosc;
        // Wykonujemy przekierowanie HTTP na podany URL.
        header("Location: " . $przekierowanie);
        // Zakończenie wykonywania bieżącego skryptu po przekierowaniu jest ważne, aby zapobiec dalszemu ładowaniu strony.
        exit;
    }
}
```



```php
/**
 * Pobiera dane pracownika (np. pensję) z bazy danych na podstawie id_uzytkownika.
 * Funkcja ta wykonuje zapytanie do bazy danych w celu wyszukania informacji o pracowniku powiązanym z podanym ID użytkownika.
 *
 * @param mysqli $polaczenie Obiekt połączenia z bazą danych MySQL. Funkcja wymaga aktywnego połączenia z bazą.
 * @param int $id_uzytkownika Unikalny identyfikator użytkownika, który jest powiązany z danymi pracownika w tabeli 'pracownicy'.
 * @return array|null Tablica asocjacyjna zawierająca dane pracownika (nazwy kolumn jako klucze) lub null, jeśli nie znaleziono pracownika o podanym ID użytkownika.
 */
function pobierz_dane_pracownika($polaczenie, $id_uzytkownika) {
    // Definiowanie zapytania SQL do pobrania wszystkich kolumn z tabeli 'pracownicy' dla wiersza, gdzie 'id_uzytkownika' odpowiada podanej wartości.
    $sql = "SELECT * FROM pracownicy WHERE id_uzytkownika = ?";
    // Przygotowanie zapytania SQL do wykonania. Prepared statements są bezpieczniejsze i wydajniejsze.
    $stmt = $polaczenie->prepare($sql);
    // Sprawdzenie, czy przygotowanie zapytania powiodło się. Jeśli nie, wyświetl błąd i zakończ skrypt.
    if (!$stmt) {
        die("Błąd zapytania: " . $polaczenie->error);
    }
    // Powiązanie parametru (id_uzytkownika) z przygotowanym zapytaniem. "i" oznacza, że parametr jest typu integer.
    $stmt->bind_param("i", $id_uzytkownika);
    // Wykonanie przygotowanego zapytania.
    $stmt->execute();
    // Pobranie wyniku zapytania.
    $result = $stmt->get_result();
    // Sprawdzenie, czy zapytanie zwróciło dokładnie jeden wiersz (oczekiwane dla unikalnego id_uzytkownika).
    if ($result->num_rows === 1) {
        // Jeśli znaleziono jednego pracownika, pobierz jego dane jako tablicę asocjacyjną (klucze to nazwy kolumn).
        return $result->fetch_assoc();
    } else {
        // Jeśli nie znaleziono żadnego pracownika lub znaleziono więcej niż jednego (co nie powinno się zdarzyć przy poprawnym ID), zwróć null.
        return null;
    }
}
?>
```

6.2. Pobieranie danych z bazy (`produkty`, `opinie`, `zamówienia`)

Dane są dynamicznie pobierane z tabeli `produkty` i prezentowane na stronie głównej oraz podstronach. Opinie klientów z tabeli `opinie` są powiązane z produktami (`id_produktu`) i mogą być prezentowane z oceną i komentarzem. Widoki SQL (np. `widok_najlepsze_produkty`) wspierają statystyki.

```php
<?php
session_start(); // Rozpoczęcie sesji
include 'auth_utils.php'; // Dołączenie pliku z funkcjami uwierzytelniania

// Sprawdzenie, czy użytkownik jest zalogowany
$zalogowany = isset($_SESSION['username']);
$rola = $_SESSION['rola'] ?? 'gość';  // Ustawienie domyślnej roli jako "gość"

// Dane logowania do bazy danych
$host = "localhost";
$uzytkownik_db = "root";
$haslo_db = "";
$nazwa_bazy = "buty";

// Nawiązanie połączenia z bazą danych
$polaczenie = new mysqli($host, $uzytkownik_db, $haslo_db, $nazwa_bazy);

if ($polaczenie->connect_error) {
    die("Błąd połączenia z bazą danych: " . $polaczenie->connect_error);
}

// Informacje o produkcie (stałe ID i dane produktu)
$product_id = 1;
$product_name = "Nike Air Force 1 Białe";
$product_price = 499.00;
$product_image = "img/Nike/AF1/AF1white.jpg";

// Funkcja do pobierania opinii z bazy danych
function pobierz_opinie($polaczenie, $id_produktu) {
    $opinie = [];
    $sql = "SELECT ocena, komentarz, imie FROM opinie WHERE id_produktu = ?";
    $stmt = $polaczenie->prepare($sql);
    $stmt->bind_param("i", $id_produktu);
    $stmt->execute();
    $result = $stmt->get_result();

    while ($row = $result->fetch_assoc()) {
        $opinie[] = $row; // Dodanie każdej opinii do tablicy
    }
    $stmt->close();
    return $opinie;
}
```

6.3. Obsługa formularza kontaktowego

Formularz kontaktowy zapisuje dane do tabeli `wiadomosci`, zawierającej pola: `imie`, `email`, `pytanie`. Dane nie są walidowane pod kątem poprawności e-maila ani długości tekstu (do poprawy). Formularz umożliwia klientom zgłaszanie problemów i pytań.

```php
<?php
session_start(); // Uruchomienie sesji
include 'auth_utils.php'; // Dołączenie pliku z funkcjami autoryzacji

// Sprawdzenie, czy użytkownik jest zalogowany i jaka jest jego rola
$zalogowany = isset($_SESSION['username']);
$rola = $_SESSION['rola'] ?? 'gość';  // Jeśli nie podano roli, przypisz 'gość'

// Obsługa formularza po jego wysłaniu (metoda POST)
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $con = mysqli_connect("localhost", "root", "", "buty"); // Połączenie z bazą danych

    // Sprawdzenie połączenia z bazą
    if (!$con) {
        $_SESSION['komunikat'] = "Błąd połączenia z bazą danych.";
        header("Location: " . $_SERVER['REQUEST_URI']);
        exit;
    }

    // Oczyszczenie danych z formularza przed zapisaniem do bazy
    $imie = mysqli_real_escape_string($con, $_POST["imie"]);
    $email = mysqli_real_escape_string($con, $_POST["email"]);
    $pytanie = mysqli_real_escape_string($con, $_POST["pytanie"]);

    // Zapytanie SQL - dodanie wiadomości do bazy danych
    $insert_query = "INSERT INTO wiadomosci (imie, email, pytanie)
                     VALUES ('$imie', '$email', '$pytanie')";

    // Sprawdzenie, czy zapytanie zostało wykonane
    if (mysqli_query($con, $insert_query)) {
        $_SESSION['komunikat'] = 'Wiadomość została wysłana! Dziękujemy za kontakt 😊';
    } else {
        $_SESSION['komunikat'] = 'Błąd podczas wysyłania wiadomości.';
    }

    mysqli_close($con); // Zamknięcie połączenia
    header("Location: " . $_SERVER['REQUEST_URI']); // Odświeżenie strony po wysłaniu
    exit;
}
?>
```

```php
<?php
// Sprawdzenie czy jest komunikat i wyświetlenie go jako "toast"
if (!empty($_SESSION['komunikat'])) {
    $komunikat = $_SESSION['komunikat'];
    $klasa = (strpos($komunikat, 'Błąd') !== false) ? 'blad' : '';

    echo "<script>
        document.addEventListener('DOMContentLoaded', function() {
            const msg = document.getElementById('komunikat');
            msg.className = '$klasa';
            msg.innerText = " . json_encode($komunikat) . ";
            msg.style.display = 'block';
            setTimeout(() => { msg.style.display = 'none'; }, 4000);
        });
    </script>";

    unset($_SESSION['komunikat']); // Wyczyszczenie komunikatu
}
?>
```

6.4. Panel administracyjny (dodawanie/edycja/usuwanie wpisów)

Pliki PHP (`dodaj_produkt.php`, `edytuj_pracownika.php`, itp.) umożliwiają pełne zarządzanie zawartością bazy:

- dodawanie produktów (do `produkty`), edycję pracowników
- (`pracownicy`), wgląd do zamówień (`zamowienia`,
- `elementy_zamowienia`), dostęp do wiadomości klientów
- (`wiadomosci`)

Moderuj_opinie.php

```php
<?php
session_start();
include 'auth_utils.php';

if (!czy_ma_role(['admin', 'szef'])) {
    header("Location: index.php");
    exit;
}

$conn = new mysqli("localhost", "root", "", "buty");
if ($conn->connect_error) {
    die("Błąd połączenia z bazą danych: " . $conn->connect_error);
}

// Obsługa usuwania opinii
if ($_SERVER["REQUEST_METHOD"] == "POST" && isset($_POST['usun_opinie'])) {
    $id_opinii = $_POST['id_opinii'];

    $sql = "DELETE FROM opinie WHERE id_opinii = $id_opinii";
    if ($conn->query($sql) === TRUE) {
        $_SESSION['komunikat'] = "Opinia została usunięta.";
        header("Location: moderuj_opinie.php");
        exit;
    } else {
        $error = "Błąd usuwania opinii: " . $conn->error;
    }
}

// Pobierz listę opinii
$sql = "SELECT o.id_opinii, o.komentarz, o.data_opinii, o.imie, p.nazwa AS nazwa_produktu
        FROM opinie o
        JOIN produkty p ON o.id_produktu = p.id_produktu
        ORDER BY o.data_opinii DESC";
$result = $conn->query($sql);

?>
```

```php
<?php
if ($result->num_rows > 0) {
    while ($row = $result->fetch_assoc()) {
        echo "<tr>";
        echo "<td>" . $row["id_opinii"] . "</td>";
        echo "<td>" . $row["nazwa_produktu"] . "</td>";
        echo "<td>" . $row["imie"] . "</td>"; // Użyj imie z tabeli opinie
        echo "<td>" . $row["komentarz"] . "</td>";  // Użyj komentarz zamiast tresc
        echo "<td>" . $row["data_opinii"] . "</td>";
        echo "<td>
                <form method='post' style='display:inline-block;'>
                    <input type='hidden' name='id_opinii' value='" . $row["id_opinii"] . "'>
                    <button type='submit' name='usun_opinie' class='btn btn-danger btn-sm' onclick=\"return confirm('Czy na pewno chcesz usunąć tę opinię?')\">Usuń</button>
                </form>
            </td>";
        echo "</tr>";
    }
} else {
    echo "<tr><td colspan='6'>Brak opinii do wyświetlenia.</td></tr>";
}
?>
```

Usun_pracownika.php

```php
<?php
session_start();
include 'auth_utils.php';

if (!czy_ma_role(['admin', 'szef'])) {
    header("Location: index.php");
    exit;
}

$conn = new mysqli("localhost", "root", "", "buty");
if ($conn->connect_error) {
    die("Błąd połączenia z bazą danych: " . $conn->connect_error);
}

if ($_SERVER["REQUEST_METHOD"] == "POST" && isset($_POST['id_pracownika'])) {
    $id_pracownika = $_POST['id_pracownika'];

    $sql = "DELETE FROM pracownicy WHERE id_pracownika = $id_pracownika";
    if ($conn->query($sql) === TRUE) {
        $_SESSION['komunikat'] = "Pracownik został usunięty.";
    } else {
        $_SESSION['error'] = "Błąd usuwania pracownika: " . $conn->error;
    }
}

header("Location: pracownicy.php"); // Powrót do listy pracowników
exit;

$conn->close();
?>
```

Zarzadzaj_pracownikami.php

```php
<?php
session_start();
include 'auth_utils.php';

if (!czy_ma_role(['admin', 'szef'])) {
    header("Location: index.php");
    exit;
}

$conn = new mysqli("localhost", "root", "", "buty");
if ($conn->connect_error) {
    die("Błąd połączenia z bazą danych: " . $conn->connect_error);
}

// Pobierz listę pracowników
$sql = "SELECT id_pracownika, nazwa_uzytkownika, email, stanowisko, pensja FROM pracownicy";
$result = $conn->query($sql);

?>
```

```php
<?php
if ($result->num_rows > 0) {
    while ($row = $result->fetch_assoc()) {
        echo "<tr>";
        echo "<td>" . $row["id_pracownika"] . "</td>";
        echo "<td>" . $row["nazwa_uzytkownika"] . "</td>";
        echo "<td>" . $row["email"] . "</td>";
        echo "<td>" . $row["stanowisko"] . "</td>";
        echo "<td>" . $row["pensja"] . "</td>";
        echo "<td>
                <a href='edytuj_pracownika_admin.php?id=" . $row["id_pracownika"] . "' class='btn btn-primary btn-sm'>Edytuj</a>
                <form method='post' action='usun_pracownika_admin.php' style='display:inline-block;'>
                    <input type='hidden' name='id_pracownika' value='" . $row["id_pracownika"] . "'>
                    <button type='submit' class='btn btn-danger btn-sm' onclick=\"return confirm('Czy na pewno chcesz usunąć tego pracownika?')\">Usuń</button>
                </form>
            </td>";
        echo "</tr>";
    }
} else {
    echo "<tr><td colspan='6'>Brak pracowników w bazie danych.</td></tr>";
}
?>
```

Edytuj_pracownika.php

```php
<?php if ($error): ?>
    <div class="alert alert-danger"><?= $error ?></div>
<?php endif; ?>
<form method="post">
    <input type="hidden" name="id_pracownika" value="<?= $id_pracownika ?>">
    <div class="mb-3">
        <label for="nazwa_uzytkownika" class="form-label">Nazwa Użytkownika</label>
        <input type="text" class="form-control" id="nazwa_uzytkownika" name="nazwa_uzytkownika" value="<?= $nazwa_uzytkownika ?>" required>
    </div>
    <div class="mb-3">
        <label for="email" class="form-label">Email</label>
        <input type="email" class="form-control" id="email" name="email" value="<?= $email ?>" required>
    </div>
    <div class="mb-3">
        <label for="stanowisko" class="form-label">Stanowisko</label>
        <input type="text" class="form-control" id="stanowisko" name="stanowisko" value="<?= $stanowisko ?>" required>
    </div>
    <div class="mb-3">
        <label for="pensja" class="form-label">Pensja</label>
        <input type="number" class="form-control" id="pensja" name="pensja" value="<?= $pensja ?>" required>
    </div>
    <button type="submit" class="btn btn-primary">Zapisz</button>
    <a href="pracownicy.php" class="btn btn-secondary">Anuluj</a>
```

```php
<?php
// Rozpoczęcie sesji, umożliwiającej dostęp do zmiennych sesyjnych
session_start();
// Załączenie pliku z funkcjami uwierzytelniającymi
include 'auth_utils.php';

// Sprawdzenie, czy użytkownik ma rolę 'kierownik', 'admin' lub 'szef'
if (!czy_ma_role(['kierownik', 'admin', 'szef'])) {
    // Jeśli użytkownik nie ma wymaganej roli, zostaje przekierowany na stronę index.php
    header("Location: index.php");
    // Zakończenie wykonywania bieżącego skryptu
    exit;
}

// Utworzenie nowego połączenia z bazą danych MySQL
$conn = new mysqli("localhost", "root", "", "buty");
// Sprawdzenie, czy wystąpił błąd podczas połączenia z bazą danych
if ($conn->connect_error) {
    // Jeśli połączenie nie powiodło się, wyświetlenie komunikatu o błędzie i zakończenie skryptu
    die("Błąd połączenia z bazą danych: " . $conn->connect_error);
}
```

```php
// Inicjalizacja zmiennej na komunikaty o błędach
$error = "";
// Sprawdzenie, czy żądanie zostało wysłane metodą POST (po kliknięciu "Zapisz")
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    // Pobranie danych z formularza i przypisanie ich do zmiennych
    $id_pracownika = $_POST['id_pracownika'];
    $nazwa_uzytkownika = $_POST['nazwa_uzytkownika'];
    $email = $_POST['email'];
    $stanowisko = $_POST['stanowisko'];
    $pensja = $_POST['pensja'];

    // Zapytanie SQL do zaktualizowania danych pracownika w tabeli 'pracownicy'
    $sql = "UPDATE pracownicy SET
                nazwa_uzytkownika='$nazwa_uzytkownika',
                email='$email',
                stanowisko='$stanowisko',
                pensja='$pensja'
                WHERE id_pracownika='$id_pracownika'";

    // Wykonanie zapytania SQL
    if ($conn->query($sql) === TRUE) {
        // Jeśli aktualizacja przebiegła pomyślnie, przekierowanie na stronę z listą pracowników
        header("Location: pracownicy.php");
        // Zakończenie wykonywania bieżącego skryptu
        exit;
    } else {
        // Jeśli wystąpił błąd podczas aktualizacji, zapisanie komunikatu o błędzie
        $error = "Błąd edycji pracownika: " . $conn->error;
    }
}
```

```php
// Pobranie danych pracownika do wypełnienia formularza edycji
if (isset($_GET['id'])) {
    // Pobranie ID pracownika z parametru GET
    $id_pracownika = $_GET['id'];
    // Zapytanie SQL do pobrania danych pracownika o podanym ID
    $sql = "SELECT id_pracownika, nazwa_uzytkownika, email, stanowisko, pensja FROM pracownicy WHERE id_pracownika='$id_pracownika'";
    // Wykonanie zapytania SQL
    $result = $conn->query($sql);
    // Sprawdzenie, czy zapytanie zwróciło dokładnie jeden wiersz (oczekiwane dla unikalnego ID pracownika)
    if ($result->num_rows == 1) {
        // Pobranie wiersza wynikowego jako asocjacyjnej tablicy
        $row = $result->fetch_assoc();
        // Przypisanie wartości z tablicy do odpowiednich zmiennych, które zostaną użyte w formularzu
        $nazwa_uzytkownika = $row['nazwa_uzytkownika'];
        $email = $row['email'];
        $stanowisko = $row['stanowisko'];
        $pensja = $row['pensja'];
    } else {
        // Jeśli nie znaleziono pracownika o podanym ID, przekierowanie na stronę z listą pracowników
        header("Location: pracownicy.php");
        // Zakończenie wykonywania bieżącego skryptu
        exit;
    }
} else {
    // Jeśli nie przekazano ID pracownika w parametrze GET, przekierowanie na stronę z listą pracowników
    header("Location: pracownicy.php");
    // Zakończenie wykonywania bieżącego skryptu
    exit;
}
?>
```

Dodaj_pracownika.php

```php
<?php
session_start();
include 'auth_utils.php';

if (!czy_ma_role(['kierownik', 'admin', 'szef'])) {
    header("Location: index.php");
    exit;
}

$conn = new mysqli("localhost", "root", "", "buty");
if ($conn->connect_error) {
    die("Błąd połączenia z bazą danych: " . $conn->connect_error);
}

$error = "";
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $nazwa_uzytkownika = $_POST['nazwa_uzytkownika'];
    $haslo = $_POST['haslo'];
    $email = $_POST['email'];
    $stanowisko = $_POST['stanowisko'];
    $pensja = $_POST['pensja'];
    $data_zatrudnienia = $_POST['data_zatrudnienia']; // Pobranie daty zatrudnienia

    // Zabezpieczenie przed SQL Injection - UŻYWAJ PREPARED STATEMENTS!
    $sql = "INSERT INTO pracownicy (nazwa_uzytkownika, haslo, email, stanowisko, pensja, data_zatrudnienia)
            VALUES (?, ?, ?, ?, ?, ?)";
    $stmt = $conn->prepare($sql);
    $stmt->bind_param("ssssds", $nazwa_uzytkownika, $haslo, $email, $stanowisko, $pensja, $data_zatrudnienia);

    if ($stmt->execute()) {
        header("Location: pracownicy.php"); // Przekierowanie po dodaniu
        exit;
    } else {
        $error = "Błąd dodawania pracownika: " . $stmt->error;
    }

    $stmt->close();
}
?>
```

```php
<div class="container">
    <div class="card">
        <div class="card-header">
            <h2>Dodaj Pracownika</h2>
        </div>
        <div class="card-body">
            <?php if ($error): ?>
                <div class="alert alert-danger"><?= $error ?></div>
            <?php endif; ?>
            <form method="post">
                <div class="mb-3">
                    <label for="nazwa_uzytkownika" class="form-label">Nazwa Użytkownika</label>
                    <input type="text" class="form-control" id="nazwa_uzytkownika" name="nazwa_uzytkownika" required>
                </div>
                <div class="mb-3">
                    <label for="haslo" class="form-label">Hasło</label>
                    <input type="password" class="form-control" id="haslo" name="haslo" required>
                </div>
                <div class="mb-3">
                    <label for="email" class="form-label">Email</label>
                    <input type="email" class="form-control" id="email" name="email" required>
                </div>
                <div class="mb-3">
                    <label for="stanowisko" class="form-label">Stanowisko</label>
                    <input type="text" class="form-control" id="stanowisko" name="stanowisko" required>
                </div>
                <div class="mb-3">
                    <label for="pensja" class="form-label">Pensja</label>
                    <input type="number" class="form-control" id="pensja" name="pensja" required>
                </div>
                <div class="mb-3">
                    <label for="data_zatrudnienia" class="form-label">Data Zatrudnienia</label>
                    <input type="date" class="form-control" id="data_zatrudnienia" name="data_zatrudnienia" required>
                </div>
                <button type="submit" class="btn btn-primary">Dodaj</button>
                <a href="pracownicy.php" class="btn btn-secondary">Anuluj</a>
            </form>
        </div>
    </div>
</div>
```

7. Baza danych

7.1. Projekt struktury bazy danych

Baza `buty` zawiera kompletny system e-commerce z produktami, koszykami, zamówieniami, opiniami i użytkownikami. Wspiera rozdział ról między klientami i administracją.

System oparty jest na relacyjnej bazie danych MySQL, która zawiera tabele obsługujące:

- klientów (`klienci`), pracowników (`pracownicy`),
- produkty (`produkty`), zamówienia (`zamowienia`,
-
-
-
-
-

`elementy_zamowienia`), opinie (`opinie`), koszyki (`koszyki`), wiadomości (`wiadomosci`).



Table	Action	Rekordy	Typ	Collation	Rozmiar	Nadmiar
elementy_zamowienia	Browse Structure Szukaj Wstaw Empty Drop	43	InnoDB	utf8mb4_general_ci	64.0 KB	-
klienci	Browse Structure Szukaj Wstaw Empty Drop	28	InnoDB	utf8mb4_general_ci	48.0 KB	-
koszyki	Browse Structure Szukaj Wstaw Empty Drop	4	InnoDB	utf8mb4_general_ci	48.0 KB	-
opinie	Browse Structure Szukaj Wstaw Empty Drop	11	InnoDB	utf8mb4_general_ci	32.0 KB	-
pracownicy	Browse Structure Szukaj Wstaw Empty Drop	9	InnoDB	utf8mb4_general_ci	48.0 KB	-
produkty	Browse Structure Szukaj Wstaw Empty Drop	24	InnoDB	utf8mb4_general_ci	16.0 KB	-
wiadomosci	Browse Structure Szukaj Wstaw Empty Drop	15	InnoDB	utf8mb4_general_ci	16.0 KB	-
widok_najaktywniejsi_klienci	Browse Structure Szukaj Wstaw Edit Drop	~0	Widok	---	-	-
widok_najlepsze_produkty	Browse Structure Szukaj Wstaw Edit Drop	~0	Widok	---	-	-
widok_opinie_z_danymi	Browse Structure Szukaj Wstaw Edit Drop	~0	Widok	---	-	-
widok_pele_zamowienia	Browse Structure Szukaj Wstaw Edit Drop	~0	Widok	---	-	-
widok_produkty_wedlug_kategorii	Browse Structure Szukaj Wstaw Edit Drop	~0	Widok	---	-	-
widok_statystyki_produktow	Browse Structure Szukaj Wstaw Edit Drop	~0	Widok	---	-	-
widok_zamowienia_z_klientami	Browse Structure Szukaj Wstaw Edit Drop	~0	Widok	---	-	-
wydatki	Browse Structure Szukaj Wstaw Empty Drop	16	InnoDB	utf8mb4_general_ci	16.0 KB	-
zamowienia	Browse Structure Szukaj Wstaw Empty Drop	38	InnoDB	utf8mb4_general_ci	32.0 KB	-
16 tables	Suma	~188	InnoDB	utf8mb4_general_ci	320.0 KB	0 B

7.2. Opis tabel i pól:

Tabela `produkty`

Pole	Typ	Opis
id_produktu	int	Klucz główny
nazwa	varchar(255)	Nazwa buta
opis	text	Opis produktu
cena	decimal(10,2)	Cena
marka	varchar(100)	Producent (np. Nike)
kategoria	varchar(100)	Typ (np. trampki, klapki)

url_zdjecia	varchar(255)	Ścieżka do zdjęcia

```
--
-- Struktura tabeli dla tabeli `produkty`
--

CREATE TABLE `produkty` (
  `id_produktu` int(11) NOT NULL,
  `nazwa` varchar(255) NOT NULL,
  `opis` text DEFAULT NULL,
  `cena` decimal(10,2) NOT NULL,
  `marka` varchar(100) DEFAULT NULL,
  `kategoria` varchar(100) DEFAULT NULL,
  `url_zdjecia` varchar(255) DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

--
-- Dumping data for table `produkty`
--
```

Tabela `klienci`

Pole	Typ	Opis
id_klienta	int	Klucz główny
nazwa_uzytkownika	varchar(50)	Login
haslo	varchar(255)	Hasło
email	varchar(100)	Adres e-mail
data_rejestracji	timestamp	Data rejestracji
rola	varchar(20)	klient

```sql
--
-- Struktura tabeli dla tabeli `klienci`
--

CREATE TABLE `klienci` (
  `id_klienta` int(11) NOT NULL,
  `nazwa_uzytkownika` varchar(50) NOT NULL,
  `haslo` varchar(255) NOT NULL,
  `email` varchar(100) NOT NULL,
  `data_rejestracji` timestamp NOT NULL DEFAULT current_timestamp(),
  `rola` varchar(20) NOT NULL DEFAULT 'klient'
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

--
-- Dumping data for table `klienci`
--
```

Tabela pracownik

Pole	Typ	Opis
id_pracownika	int	Klucz główny
nazwa_uzytkownika	varchar(50)	Login pracownika
haslo	varchar(255)	Hasło (plaintext – do poprawy)
email	varchar(100)	E-mail pracownika
data_zatrudnienia	date	Data zatrudnienia
stanowisko	varchar(50)	Rola (admin, kierownik, szef Pracownik sklepu.)
pensja	decimal(10,2)	Wysokość pensji

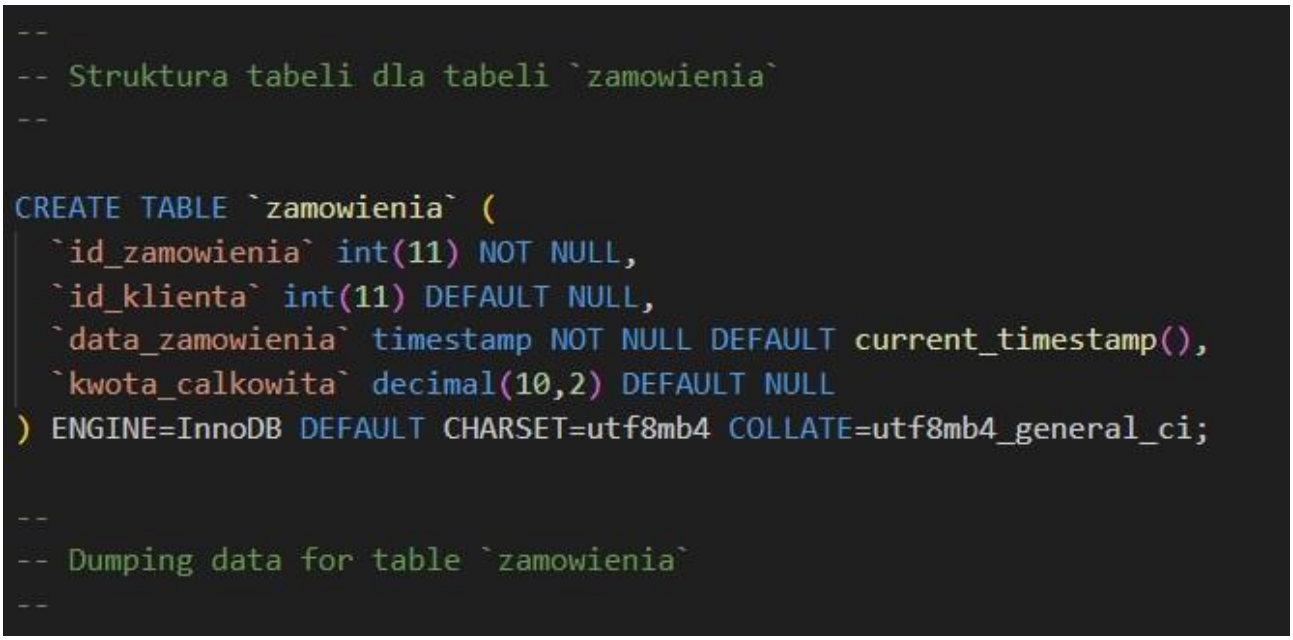
```sql
--
-- Struktura tabeli dla tabeli `pracownicy`
--

CREATE TABLE `pracownicy` (
  `id_pracownika` int(11) NOT NULL,
  `nazwa_uzytkownika` varchar(50) NOT NULL,
  `haslo` varchar(255) NOT NULL,
  `email` varchar(100) NOT NULL,
  `data_zatrudnienia` date DEFAULT NULL,
  `stanowisko` varchar(50) DEFAULT NULL,
  `pensja` decimal(10,2) DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

--
-- Dumping data for table `pracownicy`
--
```

Tabela zamoówienia

Pole	Typ	Opis
`id_zamowienia`	int	Klucz główny
`id_klienta`	int	Klucz obcy → `klienci.id_klienta`
`data_zamowienia`	timestamp	Data i czas zamówienia
`kwota_calkowita`	decimal(10,2)	Suma wartości całego zamówienia



```
--
-- Struktura tabeli dla tabeli `zamowienia`
--

CREATE TABLE `zamowienia` (
  `id_zamowienia` int(11) NOT NULL,
  `id_klienta` int(11) DEFAULT NULL,
  `data_zamowienia` timestamp NOT NULL DEFAULT current_timestamp(),
  `kwota_calkowita` decimal(10,2) DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

--
-- Dumping data for table `zamowienia`
--
```

Tabela elementy_zamowienia

Pole	Typ	Opis
id_elementu_zamowienia	int	Klucz główny
id_zamowienia	int	Klucz obcy → zamowienia.id_zamowienia
id_produktu	int	Klucz obcy → produkty.id_produktu
ilosc	int	Ilość sztuk danego produktu
cena_jednostkowa	decimal(10,2)	Cena za jedną sztukę
id_klienta	int	Klucz obcy → klienci.id_klienta
rozmiar	varchar(50)	Rozmiar zamawianego obuwia

```sql
--
-- Struktura tabeli dla tabeli `elementy_zamowienia`
--

CREATE TABLE `elementy_zamowienia` (
  `id_elementu_zamowienia` int(11) NOT NULL,
  `id_zamowienia` int(11) DEFAULT NULL,
  `id_produktu` int(11) DEFAULT NULL,
  `ilosc` int(11) NOT NULL,
  `cena_jednostkowa` decimal(10,2) NOT NULL,
  `id_klienta` int(11) DEFAULT NULL,
  `rozmiar` varchar(50) DEFAULT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

--
-- Dumping data for table `elementy_zamowienia`
--
```

\

Tabela koszyki

Pole	Typ	Opis
`id_koszyka`	int	Klucz główny
`id_klienta`	int	Klucz obcy → `klienci.id_klienta`
`id_produktu`	int	Klucz obcy → `produkty.id_produktu`
`rozmiar`	varchar(50)	Rozmiar obuwia
`ilosc`	int	Ilość danego produktu w koszyku

```sql
--
-- Struktura tabeli dla tabeli `koszyki`
--

CREATE TABLE `koszyki` (
  `id_koszyka` int(11) NOT NULL,
  `id_klienta` int(11) DEFAULT NULL,
  `id_produktu` int(11) DEFAULT NULL,
  `rozmiar` varchar(50) DEFAULT NULL,
  `ilosc` int(11) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

--
-- Dumping data for table `koszyki`
--
```

Tabela opinie

Pole	Typ	Opis
`id_opinii`	int	Klucz główny
`id_produktu`	int	Klucz obcy → `produkty.id_produktu`
`imie`	varchar(50)	Imię osoby wystawiającej opinię
`ocena`	int	Ocena (np. 1–5)
`komentarz`	text	Opinia tekstowa
`data_opinii`	timestamp	Data i czas dodania opinii

```sql
--
-- Struktura tabeli dla tabeli `opinie`
--

CREATE TABLE `opinie` (
  `id_opinii` int(11) NOT NULL,
  `id_produktu` int(11) DEFAULT NULL,
  `imie` varchar(50) NOT NULL,
  `ocena` int(11) DEFAULT NULL,
  `komentarz` text DEFAULT NULL,
  `data_opinii` timestamp NOT NULL DEFAULT current_timestamp()
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

--
-- Dumping data for table `opinie`
--
```

Tabela wiadomosci

Pole	Typ	Opis
id	int	Klucz główny
imie	varchar(20)	Imię osoby kontaktującej się
email	varchar(50)	Adres e-mail
pytanie	varchar(500)	Treść pytania

```sql
--
-- Struktura tabeli dla tabeli `wiadomosci`
--

CREATE TABLE `wiadomosci` (
  `id` int(11) NOT NULL,
  `imie` varchar(20) NOT NULL,
  `email` varchar(50) NOT NULL,
  `pytanie` varchar(500) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;

--
-- Dumping data for table `wiadomosci`
--
```

Widok_najaktywniejsi_klienci



#	Nazwa	Typ	Collation	Attributes	Null	Default	Comments	Extra
1	id_klienta	int(11)			Nie	0		
2	nazwa_uzytkownika	varchar(50)	utf8mb4_general_ci		Nie	*Brak*		
3	email	varchar(100)	utf8mb4_general_ci		Nie	*Brak*		
4	liczba_zamowien	bigint(21)			Nie	0		
5	laczna_wartosc	decimal(32,2)			Tak	*NULL*		

Widok: widok_najlepsze_produkty



#	Nazwa	Typ	Collation	Attributes	Null	Default	Comments	Extra
1	id_produktu	int(11)			Nie	0		
2	nazwa	varchar(255)	utf8mb4_general_ci		Nie	*Brak*		
3	marka	varchar(100)	utf8mb4_general_ci		Tak	*NULL*		
4	liczba_zamowien	bigint(21)			Nie	0		
5	laczna_ilosc	decimal(32,0)			Tak	*NULL*		
6	laczna_wartosc	decimal(42,2)			Tak	*NULL*		

Widok_opinie_z_danymi

Browse | Structure | SQL | Szukaj | Wstaw | Export | Uprawnienia | Operacje

#	Nazwa	Typ	Collation	Attributes	Null	Default	Comments	Extra
1	id_opinii	int(11)			Nie	0		
2	ocena	int(11)			Tak	NULL		
3	komentarz	text	utf8mb4_general_ci		Tak	NULL		
4	data_opinii	timestamp			Nie	current_timestamp()		
5	klient	varchar(50)	utf8mb4_general_ci		Nie	Brak		
6	produkt	varchar(255)	utf8mb4_general_ci		Nie	Brak		
7	marka	varchar(100)	utf8mb4_general_ci		Tak	NULL		

Check all Z zaznaczonymi: Browse

Widok: widok_pele_zamowienia

#	Nazwa	Typ	Collation	Attributes	Null	Default	Comments	Extra
1	id_zamowienia	int(11)			Nie	0		
2	data_zamowienia	timestamp			Nie	current_timestamp()		
3	klient	varchar(50)	utf8mb4_general_ci		Tak	NULL		
4	produkt	varchar(255)	utf8mb4_general_ci		Nie	Brak		
5	ilosc	int(11)			Nie	Brak		
6	cena_jednostkowa	decimal(10,2)			Nie	Brak		
7	wartosc_czesciowa	decimal(20,2)			Nie	0.00		
8	kwota_calkowita	decimal(10,2)			Tak	NULL		

Widok: widok_produkty_wedlug_kategorii

Serwer: 127.0.0.1 » Database: buty » Widok: widok_produkty_wedlug_kategorii

Browse | Structure | SQL | Szukaj | Export | Uprawnienia | Operacje

#	Nazwa	Typ	Collation	Attributes	Null	Default	Comments	Extra
1	kategoria	varchar(100)	utf8mb4_general_ci		Tak	NULL		
2	liczba_produktow	bigint(21)			Nie	0		
3	najnizsza_cena	decimal(10,2)			Tak	NULL		
4	najwyzsza_cena	decimal(10,2)			Tak	NULL		
5	srednia_cena	decimal(14,6)			Tak	NULL		

Check all Z zaznaczonymi: Browse

Widok_statystyki_produktow

Serwer: 127.0.0.1 » Database: buty » Widok: widok_statystyki_produktow

Browse | Structure | SQL | Szukaj | Export | Uprawnienia | Operacj

#	Nazwa	Typ	Collation	Attributes	Null	Default	Comments	Extra
1	id_produktu	int(11)			Nie	0		
2	nazwa	varchar(255)	utf8mb4_general_ci		Nie	*Brak*		
3	marka	varchar(100)	utf8mb4_general_ci		Tak	*NULL*		
4	cena	decimal(10,2)			Nie	*Brak*		
5	liczba_zamowien	bigint(21)			Nie	0		
6	laczna_ilosc	decimal(32,0)			Tak	*NULL*		
7	laczna_wartosc	decimal(42,2)			Tak	*NULL*		

Widok_zamowienia_z_klientami

Browse | Structure | SQL | Szukaj | Wstaw | Export | Uprawnienia | Operacje

#	Nazwa	Typ	Collation	Attributes	Null	Default	Comments	Extra
1	id_zamowienia	int(11)			Nie	0		
2	data_zamowienia	timestamp			Nie	current_timestamp()		
3	kwota_calkowita	decimal(10,2)			Tak	*NULL*		
4	id_klienta	int(11)			Nie	0		
5	klient	varchar(50)	utf8mb4_general_ci		Nie	*Brak*		
6	email	varchar(100)	utf8mb4_general_ci		Nie	*Brak*		

Check all *Z zaznaczonymi:* Browse

7.3. Relacje między tabelami

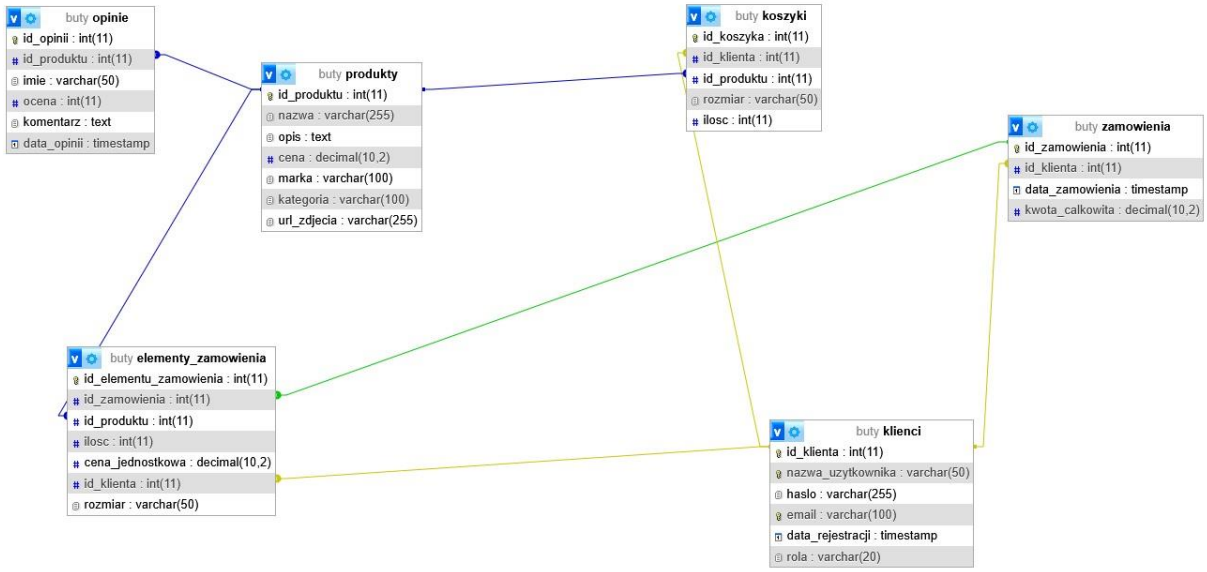
Relacje w bazie danych:

```
- Constraints for table `elementy_zamowienia`
-
LTER TABLE `elementy_zamowienia`
 ADD CONSTRAINT `elementy_zamowienia_ibfk_1` FOREIGN KEY (`id_zamowienia`) REFERENCES `zamowienia` (`id_zamowienia`),
 ADD CONSTRAINT `elementy_zamowienia_ibfk_2` FOREIGN KEY (`id_produktu`) REFERENCES `produkty` (`id_produktu`),
 ADD CONSTRAINT `fk_elementy_zamowienia_klienci` FOREIGN KEY (`id_klienta`) REFERENCES `klienci` (`id_klienta`);
-
```

```
--
-- Constraints for table `koszyki`
--
ALTER TABLE `koszyki`
  ADD CONSTRAINT `koszyki_ibfk_1` FOREIGN KEY (`id_klienta`) REFERENCES `klienci` (`id_klienta`),
  ADD CONSTRAINT `koszyki_ibfk_2` FOREIGN KEY (`id_produktu`) REFERENCES `produkty` (`id_produktu`);
```

```
--
-- Constraints for table `opinie`
--
ALTER TABLE `opinie`
  ADD CONSTRAINT `opinie_ibfk_1` FOREIGN KEY (`id_produktu`) REFERENCES `produkty` (`id_produktu`);
```

```
--
-- Constraints for table `zamowienia`
--
ALTER TABLE `zamowienia`
  ADD CONSTRAINT `zamowienia_ibfk_1` FOREIGN KEY (`id_klienta`) REFERENCES `klienci` (`id_klienta`);
COMMIT;
```



7.4. Przykładowe zapytania SQL

```
SELECT * FROM widok_najaktywniejsi_klienci;
```

Profilowanie [Edytuj w linii] [Edit] [Explain SQL] [Create PHP code] [Refresh]

> | Show all | Liczba wierszy: 25 Filter rows: Przeszukaj tę tabelę

Extra options

l_klienta	nazwa_uzytkownika	email	liczba_zamowien	laczna_wartosc
29	Andrzej	Andrzej@gmail.com	19	33709.00
5	piotr_wisniewski	piotr.wisniewski@example.com	9	16361.00
9	lukasz_nowakowski	lukasz.nowakowski@example.com	1	3198.00
30	Bombel	Bombel@gmail.com	2	1748.00
7	tomasz_muller	tomasz.muller@example.com	4	1706.00
17	sebastian_kowalczyk	sebastian.kowalczyk@example.com	2	1628.00
31	Dominik	Dominik@wp.pl	1	499.00
14	natalia_sokolowska	natalia.sokolowska@example.com	0	NULL
19	filip_zawadzki	filip.zawadzki@example.com	0	NULL
1	jan_kowalski	jan.kowalski@example.com	0	NULL
28	MarekJ	MarekJ@gmail.com	0	NULL
4	anna_smith	anna.smith@example.com	0	NULL
6	ewa_lewandowska	ewa.lewandowska@example.com	0	NULL
32	Laura	Laura123@gmail.com	0	NULL
8	magdalena_bak	magdalena.bak@example.com	0	NULL
35	Paweł	Pablo@Pablo.pl	0	NULL
11	michal_wozniak	michal.wozniak@example.com	0	NULL
38	Andrzejff	fsafsa@ggg.com	0	NULL
13	marcin_kaminski	marcin.kaminski@example.com	0	NULL
41	Oskar	oskar55@gmail.com	0	NULL
15	dawid_gorski	dawid.gorski@example.com	0	NULL
18	wiktoria_rutkowska	wiktoria.rutkowska@example.com	0	NULL
20	julia_kaczmarek	julia.kaczmarek@example.com	0	NULL
2	maria_nowak	maria.nowak@example.com	0	NULL
33	Michał	Majkel@gmail.com	0	NULL

Zapytanie SQL zostało wykonane pomyślnie.

```
SELECT COUNT(*) FROM zamowienia WHERE id_klienta = 5;
```

Profilowanie [Edytuj w linii] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Extra options

COUNT(*)
9

Operacja na wynikach zapytania

Pokazano wiersze 0 - 0 (1 total, Wykonanie zapytania trwało 0,0001 sekund(y).)

```
SELECT AVG(cena) FROM produkty WHERE marka = 'Adidas';
```

Profilowanie [Edytuj w linii] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Liczba wierszy: 25 Filter rows: Przeszukaj tę tabelę

Extra options

AVG(cena)
384.857143

Pokazano wiersze 0 - 2 (3 total, Wykonanie zapytania trwało 0,0002 sekund(y).)

```
SELECT * FROM opinie WHERE id_produktu = 1;
```

Profilowanie [Edytuj w linii] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Liczba wierszy: 25 Filter rows: Przeszukaj tę tabelę Sortuj wg klucza: Żaden

Extra options

	id_opinii	id_produktu	imie	ocena	komentarz	data_opinii
Edit Copy Delete	3	1	kaska	2	faaffsff	2025-05-17 23:04:39
Edit Copy Delete	6	1	fasfa	5	fasafsfaasfsf	2025-05-18 13:49:54
Edit Copy Delete	10	1	fsafsasfa	3	fassfsaf	2025-05-18 18:55:46

Pokazano wiersze 0 - 23 (24 total, Wykonanie zapytania trwało 0,0004 sekund(y).)

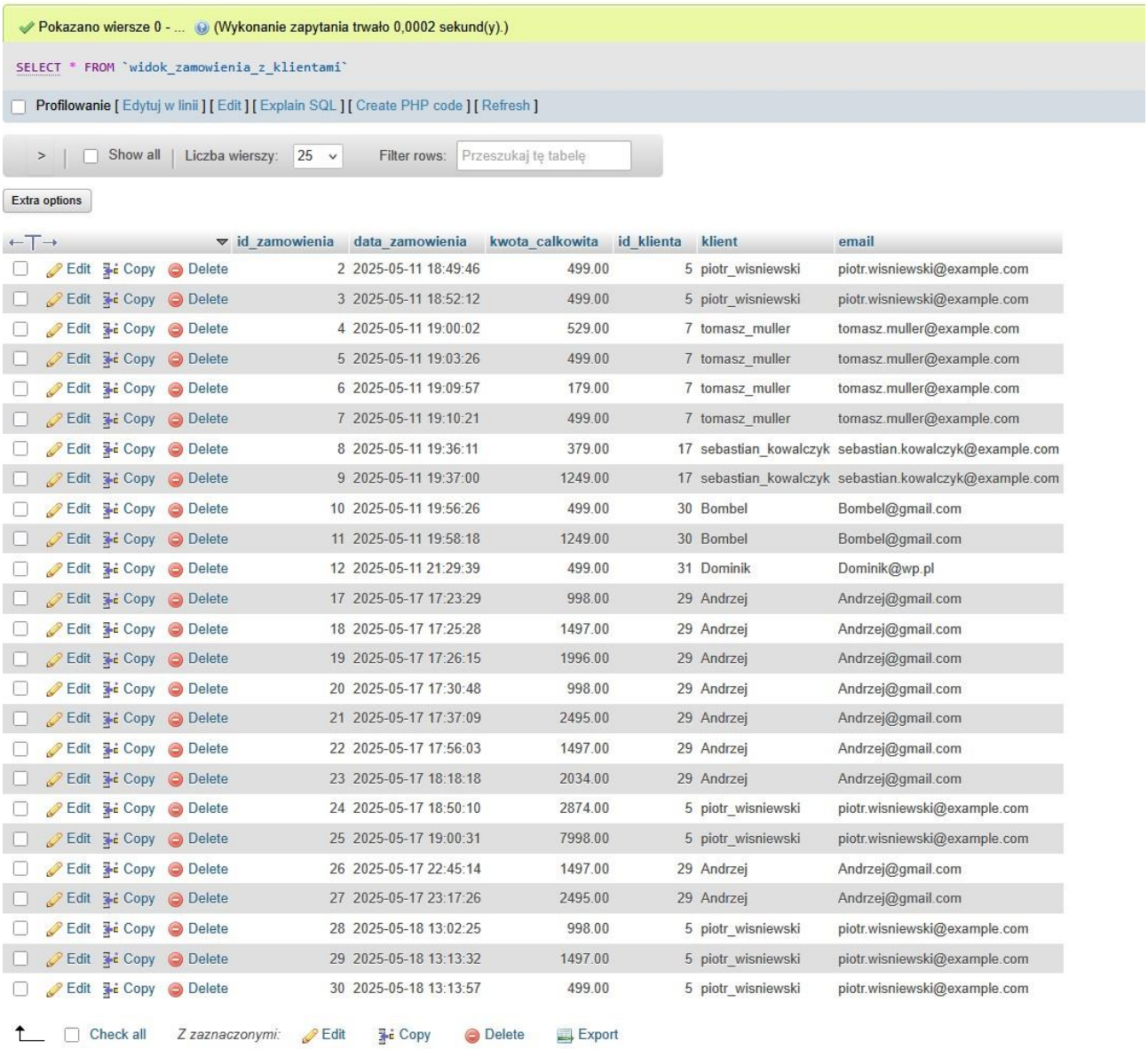
```
SELECT * FROM `widok_najlepsze_produkty`
```

Profilowanie [Edytuj w linii] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Liczba wierszy: 25 Filter rows: Przeszukaj tę tabelę

Extra options

id_produktu	nazwa	marka	liczba_zamowien	laczna_ilosc	laczna_wartosc
1	Nike Air Force 1 Białe	Nike	28	59	29441.00
2	Jordan 1 Mocha	Jordan	3	7	8743.00
7	Nike Pegasus Premium	Nike	2	4	3596.00
13	Jordan 1 Chicago	Jordan	1	2	3198.00
19	Under Armour Infinite	Under Armour	1	3	2097.00
10	Converse Chuck Taylor All Star	Converse	1	3	1047.00
23	Under Armour Magnetico	Under Armour	1	3	897.00
15	Vans Old Skool Czarny	Vans	1	2	758.00
20	Adidas Klapki Białe	Adidas	1	3	537.00
4	Adidas Campus 00s Czarne	Adidas	1	1	529.00
3	Adidas Campus 00s Beżowe	Adidas	1	1	529.00
12	Adidas Forum Low	Adidas	1	1	499.00
11	Converse All Star Platform Czarny	Converse	1	1	399.00
24	Jordan Klapki Białe	Jordan	0	*NULL*	*NULL*
17	Nike Klapki Białe	Nike	0	*NULL*	*NULL*
16	Vans Sk8-Hi Biało-Czarne	Vans	0	*NULL*	*NULL*
22	Jordan Klapki Czarne	Jordan	0	*NULL*	*NULL*
9	Reebook FIORI	Reebok	0	*NULL*	*NULL*
21	Adidas Klapki Czarne	Adidas	0	*NULL*	*NULL*
8	Nike Air Max 90	Nike	0	*NULL*	*NULL*
14	Adidas COPA PURE 2	Adidas	0	*NULL*	*NULL*
6	Jordan 4 Military Black	Jordan	0	*NULL*	*NULL*
18	Nike Klapki Czarne	Nike	0	*NULL*	*NULL*
5	Adidas Samba OG	Adidas	0	*NULL*	*NULL*



Pokazano wiersze 0 - ... (Wykonanie zapytania trwało 0,0002 sekund(y).)

```
SELECT * FROM `widok_zamowienia_z_klientami`
```

Profilowanie [Edytuj w linii] [Edit] [Explain SQL] [Create PHP code] [Refresh]

> | Show all | Liczba wierszy: 25 Filter rows: Przeszukaj tę tabelę

Extra options

	id_zamowienia	data_zamowienia	kwota_calkowita	id_klienta	klient	email
Edit Copy Delete	2	2025-05-11 18:49:46	499.00	5	piotr_wisniewski	piotr.wisniewski@example.com
Edit Copy Delete	3	2025-05-11 18:52:12	499.00	5	piotr_wisniewski	piotr.wisniewski@example.com
Edit Copy Delete	4	2025-05-11 19:00:02	529.00	7	tomasz_muller	tomasz.muller@example.com
Edit Copy Delete	5	2025-05-11 19:03:26	499.00	7	tomasz_muller	tomasz.muller@example.com
Edit Copy Delete	6	2025-05-11 19:09:57	179.00	7	tomasz_muller	tomasz.muller@example.com
Edit Copy Delete	7	2025-05-11 19:10:21	499.00	7	tomasz_muller	tomasz.muller@example.com
Edit Copy Delete	8	2025-05-11 19:36:11	379.00	17	sebastian_kowalczyk	sebastian.kowalczyk@example.com
Edit Copy Delete	9	2025-05-11 19:37:00	1249.00	17	sebastian_kowalczyk	sebastian.kowalczyk@example.com
Edit Copy Delete	10	2025-05-11 19:56:26	499.00	30	Bombel	Bombel@gmail.com
Edit Copy Delete	11	2025-05-11 19:58:18	1249.00	30	Bombel	Bombel@gmail.com
Edit Copy Delete	12	2025-05-11 21:29:39	499.00	31	Dominik	Dominik@wp.pl
Edit Copy Delete	17	2025-05-17 17:23:29	998.00	29	Andrzej	Andrzej@gmail.com
Edit Copy Delete	18	2025-05-17 17:25:28	1497.00	29	Andrzej	Andrzej@gmail.com
Edit Copy Delete	19	2025-05-17 17:26:15	1996.00	29	Andrzej	Andrzej@gmail.com
Edit Copy Delete	20	2025-05-17 17:30:48	998.00	29	Andrzej	Andrzej@gmail.com
Edit Copy Delete	21	2025-05-17 17:37:09	2495.00	29	Andrzej	Andrzej@gmail.com
Edit Copy Delete	22	2025-05-17 17:56:03	1497.00	29	Andrzej	Andrzej@gmail.com
Edit Copy Delete	23	2025-05-17 18:18:18	2034.00	29	Andrzej	Andrzej@gmail.com
Edit Copy Delete	24	2025-05-17 18:50:10	2874.00	5	piotr_wisniewski	piotr.wisniewski@example.com
Edit Copy Delete	25	2025-05-17 19:00:31	7998.00	5	piotr_wisniewski	piotr.wisniewski@example.com
Edit Copy Delete	26	2025-05-17 22:45:14	1497.00	29	Andrzej	Andrzej@gmail.com
Edit Copy Delete	27	2025-05-17 23:17:26	2495.00	29	Andrzej	Andrzej@gmail.com
Edit Copy Delete	28	2025-05-18 13:02:25	998.00	5	piotr_wisniewski	piotr.wisniewski@example.com
Edit Copy Delete	29	2025-05-18 13:13:32	1497.00	5	piotr_wisniewski	piotr.wisniewski@example.com
Edit Copy Delete	30	2025-05-18 13:13:57	499.00	5	piotr_wisniewski	piotr.wisniewski@example.com

Check all Z zaznaczonymi: Edit Copy Delete Export

8. Bezpieczeństwo

Bezpieczeństwo systemu stanowi kluczowy element przy projektowaniu i wdrażaniu aplikacji webowej. W niniejszym systemie wdrożono szereg mechanizmów zabezpieczających dane użytkowników oraz chroniących system przed najczęściej spotykanymi zagrożeniami.

8.1. Walidacja danych wejściowych

Aby zapobiec wprowadzaniu błędnych, niekompletnych lub szkodliwych danych do systemu, wdrożono walidację danych na dwóch poziomach:

Po stronie klienta (JavaScript):

- Sprawdzanie poprawności formatu adresu e-mail.

- Wymagane pola formularzy (atrybuty `required`).
- Sprawdzenie długości haseł i dopuszczalnych znaków.

- Weryfikacja długości wiadomości w formularzu kontaktowym.

Po stronie serwera (PHP):

- Sprawdzenie i oczyszczenie danych wejściowych przy użyciu funkcji `filter_input()`, `htmlspecialchars()` i `mysqli_real_escape_string()`.

- Weryfikacja poprawności danych przed zapisaniem ich do bazy danych.

- Ochrona przed atakami typu SQL Injection poprzez stosowanie zapytań z parametryzacją (przygotowane zapytania, np. `mysqli_prepare()`).

Walidacja po obu stronach systemu zapewnia większą odporność na błędy i ataki oraz poprawia komfort użytkownika.

8.2. Obsługa sesji i uwierzytelnianie

System wykorzystuje mechanizm sesji PHP do zarządzania stanem zalogowanego użytkownika oraz do kontroli dostępu do zasobów chronionych.

- Po pomyślnym zalogowaniu użytkownik otrzymuje identyfikator sesji (`$_SESSION['id_klienta']`), który umożliwia identyfikację i przypisanie odpowiednich uprawnień.

- Panel administracyjny jest dostępny wyłącznie dla użytkowników o odpowiedniej roli (np. `admin`, `kierownik`).

- Przed wykonaniem operacji wymagających autoryzacji system sprawdza, czy użytkownik jest zalogowany i posiada odpowiednie uprawnienia.

- W celu zabezpieczenia hasła stosowana jest funkcja `password_hash()` do jego haszowania, oraz `password_verify()` do sprawdzenia przy logowaniu.

- Dane sesyjne są przechowywane w sposób zabezpieczony przed nieautoryzowanym dostępem (brak dostępu do danych przez URL, ustawienie odpowiednich nagłówków).

9. Możliwości rozbudowy

W obecnym stanie system zapewnia podstawową funkcjonalność sklepu internetowego. W przyszłości istnieje wiele możliwości jego rozbudowy, które mogą znacząco zwiększyć użyteczność oraz konkurencyjność witryny.

9.1. Wyszukiwarka produktów

Wdrożenie dynamicznej wyszukiwarki umożliwiłoby użytkownikom szybkie odnalezienie produktów po nazwie, marce, kategorii lub cenie. Przykładowa implementacja mogłaby

wykorzystywać zapytania SQL z klauzulą `LIKE`, a w przyszłości także pełnotekstowe wyszukiwanie (`FULLTEXT` w MySQL) lub rozwiązania typu Elasticsearch.

9.2. System ocen i komentarzy

System umożliwia już dodawanie opinii o produktach, jednak można go rozbudować o:

- możliwość edycji/usuwania opinii przez użytkownika,
- system ocen w formie graficznej (np. gwiazdki),
- moderację komentarzy przez administratora, liczniki
- średnich ocen i ich wizualizację.

9.3. System rezerwacji

Możliwa jest rozbudowa o funkcję rezerwacji towaru z odbiorem osobistym lub tymczasowym "zarezerwowaniem" przed opłaceniem. System mógłby zawierać:

- kalendarz dostępności produktu, czasową blokadę
- produktu po dodaniu do koszyka, automatyczne
- powiadomienia e-mail o statusie rezerwacji.

9.4. Responsywność i wersja mobilna

Aby dostosować stronę do urządzeń mobilnych, można wdrożyć:

- pełną responsywność przy użyciu CSS (media queries) lub frameworka Bootstrap, mobilne
- menu, przyciski i formularze,
- Progressive Web App (PWA), umożliwiające instalację sklepu jako aplikacji mobilnej.

10. Załączniki

W celu udokumentowania działania systemu oraz ułatwienia jego analizy i modyfikacji, poniżej przedstawiono zestawienie materiałów pomocniczych.

10.1. Fragmenty kodu źródłowego z komentarzami

Do dokumentacji dołączone zostały fragmenty kodu HTML, PHP, JavaScript i CSS z komentarzami wyjaśniającymi ich działanie, m.in.:

- logika logowania i rejestracji (`logowanie.php`, `rejestracja.php`)

  Rejestracja:

```php
<?php
session_start(); // Uruchomienie mechanizmu sesji, który umożliwia przechowywanie danych o użytkowniku między żądaniami.
$zalogowany = isset($_SESSION['username']); // Sprawdzenie, czy w sesji istnieje zmienna 'username'. Jeśli tak, użytkownik jest zalogowan
?>
<?php
$host = 'localhost'; // Adres serwera bazy danych.
$uzytkownik = 'root'; // Nazwa użytkownika bazy danych.
$haslo_db = ''; // Hasło do bazy danych (w tym przypadku puste).
$nazwa_bazy = 'buty'; // Nazwa bazy danych, z którą chcemy się połączyć.

// Utworzenie nowego obiektu mysqli do połączenia z bazą danych.
$polaczenie = new mysqli($host, $uzytkownik, $haslo_db, $nazwa_bazy);

// Sprawdzenie, czy wystąpił błąd podczas łączenia z bazą danych.
if ($polaczenie->connect_error) {
    die("Błąd połączenia: " . $polaczenie->connect_error); // Wyświetlenie komunikatu o błędzie i zatrzymanie wykonywania skryptu.
}

// Sprawdzenie, czy metoda żądania HTTP to POST (czyli czy formularz został wysłany).
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $username = $_POST["username"]; // Pobranie wartości pola 'username' z formularza.
    $email = $_POST["email"]; // Pobranie wartości pola 'email' z formularza.
    $password = $_POST["password"]; // Pobranie wartości pola 'password' z formularza.

    // Zapytanie SQL do wstawienia nowego klienta do tabeli 'klienci'.
    // Użyto prepared statements, aby zapobiec atakom SQL injection.
    $sql = "INSERT INTO klienci (nazwa_uzytkownika, email, haslo) VALUES (?, ?, ?)";
    $stmt = $polaczenie->prepare($sql); // Przygotowanie zapytania SQL do wykonania.

    // Sprawdzenie, czy przygotowanie zapytania się powiodło.
    if (!$stmt) {
        die("Błąd zapytania: " . $polaczenie->error); // Wyświetlenie błędu, jeśli przygotowanie się nie powiodło.
    }
```

```php
    // Powiązanie parametrów z przygotowanym zapytaniem. "sss" oznacza, że wszystkie trzy parametry są typu string.
    $stmt->bind_param("sss", $username, $email, $password);

    // Wykonanie przygotowanego zapytania SQL.
    if ($stmt->execute()) {
        // Jeśli rejestracja przebiegła pomyślnie, wyświetl komunikat sukcesu ze stylami CSS wewnątrz znacznika <style>.
        echo '
        <style>
            .success-alert {
                max-width: 500px;
                margin: 50px auto;
                padding: 20px 25px;
                background-color: #e6ffed;
                border: 1px solid #b7eb8f;
                border-radius: 8px;
                color: #2f855a;
                font-family: Arial, sans-serif;
                font-size: 16px;
                box-shadow: 0 2px 6px rgba(0, 0, 0, 0.1);
                position: relative;
            }
            .success-alert::before {
                content: "✓";
                font-size: 20px;
                position: absolute;
                left: 15px;
                top: 18px;
            }
            .success-alert .message {
                padding-left: 30px;
            }
        </style>
        <div class="success-alert">
            <div class="message">
                Użytkownik zarejestrowany pomyślnie! Możesz się teraz <a href="login.php">zalogować</a>.
            </div>
        </div>';
    } else {
        // Jeśli wystąpił błąd podczas rejestracji, wyświetl komunikat o błędzie.
        echo "Błąd podczas rejestracji: " . $stmt->error;
    }

    // Zamknięcie przygotowanego zapytania.
    $stmt->close();
}

// Zamknięcie połączenia z bazą danych.
```

```html
  <div class="auth-container">
      <h2>Załóż konto</h2>
      <form action="register.php" method="POST">
          <input type="text" name="username" placeholder="Nazwa użytkownika" required>
          <input type="email" name="email" placeholder="Email" required>
          <input type="password" name="password" placeholder="Hasło" required>

          <button type="submit">Zarejestruj się</button>
      </form>
  </div>
</div>
```

Logowanie

```php
// Sprawdzenie, czy formularz logowania został wysłany metodą POST.
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $username = $_POST["username"]; // Pobranie nazwy użytkownika z formularza.
    $password = $_POST["password"]; // Pobranie hasła z formularza.

    // Sprawdzenie danych logowania w tabeli klientów.
    $sql_klienci = "SELECT id_klienta, nazwa_uzytkownika, haslo, rola FROM klienci WHERE nazwa_uzytkownika = ?"; // Zapytanie SQL wybierające dane klienta po nazwie użytkownika. Dodano 'rola'.
    $stmt_klienci = $polaczenie->prepare($sql_klienci); // Przygotowanie zapytania SQL.

    if ($stmt_klienci) {
        $stmt_klienci->bind_param("s", $username); // Powiązanie nazwy użytkownika z parametrem zapytania.
        $stmt_klienci->execute(); // Wykonanie zapytania.
        $wynik_klienci = $stmt_klienci->get_result(); // Pobranie wyników zapytania.

        // Sprawdzenie, czy znaleziono dokładnie jednego klienta o podanej nazwie użytkownika.
        if ($wynik_klienci->num_rows === 1) {
            $uzytkownik_klienci = $wynik_klienci->fetch_assoc(); // Pobranie wiersza wyniku jako tablicy asocjacyjnej.
            // Sprawdzenie, czy wprowadzone hasło zgadza się z hasłem w bazie danych.
            if ($password == $uzytkownik_klienci['haslo']) {
                // Ustawienie zmiennych sesji po pomyślnym zalogowaniu klienta.
                $_SESSION['username'] = htmlspecialchars($uzytkownik_klienci['nazwa_uzytkownika']); // Bezpieczne zapisanie nazwy użytkownika.
                $_SESSION['rola'] = $uzytkownik_klienci['rola']; // Pobranie i zapisanie roli klienta w sesji.
                $_SESSION['id_uzytkownika'] = $uzytkownik_klienci['id_klienta']; // Zapisanie ID klienta w sesji.
                $_SESSION['zalogowano_pomyslnie'] = true; // Flaga informująca o pomyślnym zalogowaniu.

                // Pobierz koszyk klienta z bazy danych.
                if ($_SESSION['rola'] === 'klient') { // Sprawdzenie, czy zalogowana osoba to klient.
                    $id_klienta = $_SESSION['id_uzytkownika']; // Pobranie ID klienta z sesji.
                    $sql_koszyk = "SELECT ez.*, p.cena, p.nazwa, p.url_zdjecia
                                   FROM elementy_zamowienia ez
                                   JOIN produkty p ON ez.id_produktu = p.id_produktu
                                   WHERE ez.id_klienta = ?"; // Zapytanie SQL pobierające elementy koszyka klienta.
                    $stmt_koszyk = $polaczenie->prepare($sql_koszyk); // Przygotowanie zapytania.

                    if ($stmt_koszyk) {
                        $stmt_koszyk->bind_param("i", $id_klienta); // Powiązanie ID klienta z parametrem.
                        $stmt_koszyk->execute(); // Wykonanie zapytania.
                        $wynik_koszyk = $stmt_koszyk->get_result(); // Pobranie wyników.
                        $_SESSION['koszyk'] = $wynik_koszyk->fetch_all(MYSQLI_ASSOC); // Zapisanie koszyka klienta w sesji.
                    }
                }
            } else {
                $wiadomosc_bledu = "Nieprawidłowe hasło."; // Ustawienie komunikatu o błędnym haśle.
                $wiadomosc_bledu = "Nieprawidłowe hasło."; // Ustawienie komunikatu o błędnym haśle.
            }
        } else {
            // Jeśli nie znaleziono klienta, sprawdź w tabeli pracowników.
            $sql_pracownicy = "SELECT id_pracownika, nazwa_uzytkownika, haslo, stanowisko FROM pracownicy WHERE nazwa_uzytkownika = ?"; // Zapytanie SQL dla pracowników.
            $stmt_pracownicy = $polaczenie->prepare($sql_pracownicy); // Przygotowanie zapytania.

            if ($stmt_pracownicy) {
                $stmt_pracownicy->bind_param("s", $username); // Powiązanie nazwy użytkownika.
                $stmt_pracownicy->execute(); // Wykonanie zapytania.
                $wynik_pracownicy = $stmt_pracownicy->get_result(); // Pobranie wyników.

                // Sprawdzenie, czy znaleziono pracownika.
                if ($wynik_pracownicy->num_rows === 1) {
                    $uzytkownik_pracownicy = $wynik_pracownicy->fetch_assoc(); // Pobranie danych pracownika.
                    // Sprawdzenie hasła pracownika.
                    if ($password == $uzytkownik_pracownicy['haslo']) {
                        // Ustawienie zmiennych sesji dla zalogowanego pracownika.
                        $_SESSION['username'] = htmlspecialchars($uzytkownik_pracownicy['nazwa_uzytkownika']);
                        $_SESSION['rola'] = $uzytkownik_pracownicy['stanowisko']; // Ustawienie stanowiska jako roli.
                        $_SESSION['id_uzytkownika'] = $uzytkownik_pracownicy['id_pracownika'];
                        $_SESSION['zalogowano_pomyslnie'] = true;
                        // Nie pobieramy koszyka dla pracowników.
                    } else {
                        $wiadomosc_bledu = "Nieprawidłowe hasło."; // Błędne hasło pracownika.
                    }
                } else {
                    $wiadomosc_bledu = "Nieprawidłowy użytkownik."; // Nie znaleziono użytkownika ani w klientach, ani w pracownikach.
                }
                $stmt_pracownicy->close(); // Zamknięcie zapytania dla pracowników.
            } else {
                $wiadomosc_bledu = "Błąd w zapytaniu SQL (pracownicy)."; // Błąd przygotowania zapytania dla pracowników.
            }
        }
        $stmt_klienci->close(); // Zamknięcie zapytania dla klientów.
    } else {
        $wiadomosc_bledu = "Błąd w zapytaniu SQL (klienci)."; // Błąd przygotowania zapytania dla klientów.
    }
}

$polaczenie->close(); // Zamknięcie połączenia z bazą danych.
?>
```

- obsługa formularzy i zapytań do bazy (opinie.php, `kontakt.php`)
  Opinie

```
<script>
// Obsługa asynchronicznego wysyłania formularza opinii za pomocą Fetch API
document.getElementById('formularz-opinii').addEventListener('submit', async function(e) {
    e.preventDefault(); // Zapobiega domyślnej akcji formularza, czyli przeładowaniu strony.

    const imie = document.getElementById('imie').value; // Pobranie wartości z pola imienia.
    const tresc = document.getElementById('tresc').value; // Pobranie wartości z pola treści opinii.

    // Wykonanie żądania POST do serwera (do tego samego pliku 'opinie.php') w celu dodania opinii.
    const res = await fetch('opinie.php', {
        method: 'POST', // Metoda HTTP POST, używana do wysyłania danych do serwera.
        headers: {'Content-Type': 'application/x-www-form-urlencoded'}, // Ustawienie nagłówka informującego serwer o formacie danych.
        body: `imie=${encodeURIComponent(imie)}&tresc=${encodeURIComponent(tresc)}` // Dane formularza zakodowane w formacie URL.
    });

    // Sprawdzenie, czy odpowiedź z serwera jest pomyślna (status HTTP 2xx).
    if (res.ok) {
        document.getElementById('imie').value = ''; // Wyczyszczenie pola imienia po pomyślnym wysłaniu.
        document.getElementById('tresc').value = ''; // Wyczyszczenie pola treści opinii.
        wczytajOpinie(); // Wywołanie funkcji wczytującej aktualną listę opinii z serwera.
    }
});

// Asynchroniczna funkcja do pobierania i wyświetlania opinii z serwera.
async function wczytajOpinie() {
    const res = await fetch('opinie.php'); // Wykonanie żądania GET do serwera (do tego samego pliku) w celu pobrania opinii.
    const data = await res.json(); // Parsowanie odpowiedzi z serwera jako JSON.

    const container = document.getElementById('lista-opinii'); // Pobranie elementu kontenera, w którym będą wyświetlane opinie.
    container.innerHTML = ''; // Wyczyszczenie dotychczasowej zawartości kontenera opinii.

    // Iteracja po każdej opinii pobranej z serwera.
    data.forEach(opinia => {
        const div = document.createElement('div'); // Utworzenie nowego elementu div dla każdej opinii.
        div.classList.add('opinia'); // Dodanie klasy CSS 'opinia' do nowego elementu.
        div.innerHTML = `<strong>${opinia.imie}</strong>: ${opinia.komentarz} <em>(${opinia.data_opinii})</em>`; // Wstawienie danych opinii (imię, treść, data) do HTML elementu.
        container.appendChild(div); // Dodanie utworzonego elementu opinii do kontenera na stronie.
    });
}
// Wywołanie funkcji wczytującej opinie przy pierwszym załadowaniu strony, aby wyświetlić istniejące opinie.
wczytajOpinie();
</script>
```

```
<?php
$host = "localhost"; // Adres serwera bazy danych.
$db = "buty"; // Nazwa bazy danych.
$user = "root"; // Nazwa użytkownika bazy danych.
$pass = ""; // Hasło użytkownika bazy danych.
$conn = new mysqli($host, $user, $pass, $db); // Utworzenie nowego połączenia z bazą danych MySQL.
$conn->set_charset("utf8"); // Ustawienie domyślnego zestawu znaków dla połączenia na UTF-8.

// Obsługa żądania HTTP POST (dodawanie nowej opinii).
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $imie = $_POST['imie']; // Pobranie wartości pola 'imie' z danych POST.
    $tresc = $_POST['tresc']; // Pobranie wartości pola 'tresc' z danych POST.
    $id_produktu = NULL; // ID produktu, którego dotyczy opinia. Tutaj ustawione na NULL, co może wymagać zmiany w przyszłości, jeśli opinie mają być powiązane z konkretnymi produktami.
    $ocena = NULL; // Ocena opinii. Tutaj ustawione na NULL, brak implementacji zbierania oceny w formularzu.

    // Przygotowanie zapytania SQL do wstawienia nowej opinii do tabeli 'opinie'.
    $stmt = $conn->prepare("INSERT INTO opinie (id_produktu, imie, ocena, komentarz, data_opinii) VALUES (?, ?, ?, ?, NOW())");
    // Powiązanie typów i wartości parametrów z przygotowanym zapytaniem. 's' oznacza string (tekst), 'i' integer (liczba).
    $stmt->bind_param("isis", $id_produktu, $imie, $ocena, $tresc);
    // Wykonanie przygotowanego zapytania SQL.
    $stmt->execute();
    echo "OK"; // Wysłanie prostej odpowiedzi "OK" do klienta (JavaScript), informującej o pomyślnym dodaniu opinii.
    exit; // Zakończenie wykonywania skryptu PHP po obsłużeniu żądania POST.
}

// Obsługa żądania HTTP GET (pobieranie istniejących opinii).
if ($_SERVER['REQUEST_METHOD'] === 'GET') {
    // Zapytanie SQL pobierające imię, komentarz i datę dodania opinii z tabeli 'opinie', gdzie 'id_produktu' jest NULL (prawdopodobnie opinie ogólne).
    $result = $conn->query("SELECT imie, komentarz, data_opinii FROM opinie WHERE id_produktu IS NULL");
    $opinie = []; // Inicjalizacja pustej tablicy, która będzie przechowywać pobrane opinie.
    // Iteracja po wynikach zapytania SQL.
    while ($row = $result->fetch_assoc()) {
        $opinie[] = $row; // Dodanie każdego wiersza (opinii) do tablicy $opinie jako asocjacyjna tablica.
    }
    // Zakodowanie tablicy z opiniami do formatu JSON i wysłanie jej do klienta (JavaScript).
    echo json_encode($opinie);
}
?>
```

```
<script>
  // Po załadowaniu strony – wczytaj opinie z localStorage
  document.addEventListener("DOMContentLoaded", function () {
    const zapisane = JSON.parse(localStorage.getItem("opinie")) || [];
    const kontener = document.getElementById("lista-opinii");

    // Dodaj dynamiczne opinie
    zapisane.forEach((opinia, index) => {
      const el = stworzOpiniaElement(opinia.imie, opinia.tresc, index);
      kontener.appendChild(el);
    });

    // Dodaj działanie przyciskom "Usuń" dla opinii statycznych
    document.querySelectorAll(".opinia .usun-opinie").forEach(button => {
      button.addEventListener("click", function () {
        this.parentElement.remove();
      });
    });
  });

  // Obsługa formularza dodawania opinii
  document.getElementById("formularz-opinii").addEventListener("submit", function (e) {
    e.preventDefault();

    const imie = document.getElementById("imie").value.trim();
    const tresc = document.getElementById("tresc").value.trim();

    if (imie && tresc) {
      const nowaOpinia = { imie, tresc };
      const stare = JSON.parse(localStorage.getItem("opinie")) || [];
      stare.push(nowaOpinia);
      localStorage.setItem("opinie", JSON.stringify(stare));

      const el = stworzOpiniaElement(imie, tresc, stare.length - 1);
      document.getElementById("lista-opinii").appendChild(el);
      this.reset();
    }
  });
```

```
  // Funkcja tworzy element opinii z przyciskiem usuń
  function stworzOpiniaElement(imie, tresc, index) {
    const el = document.createElement("article");
    el.classList.add("opinia");

    el.innerHTML = `
      <h3>${imie}</h3>
      <p>${tresc}</p>
      <button class="usun-opinie" hidden>Usuń</button>
    `;

    // Obsługa kliknięcia przycisku usuń
    el.querySelector(".usun-opinie").addEventListener("click", function () {
      el.remove();

      // Usunięcie z localStorage
      const opinie = JSON.parse(localStorage.getItem("opinie")) || [];
      opinie.splice(index, 1); // usuń z listy po indeksie
      localStorage.setItem("opinie", JSON.stringify(opinie));
    });

    return el;
  }
</script>
```

Zapisz Opinie

```php
<?php
// Rozpocznij sesję, aby móc korzystać ze zmiennych sesyjnych, jeśli zajdzie taka potrzeba.
session_start();
// Dołącz plik 'auth_utils.php', który prawdopodobnie zawiera różne funkcje związane z autentykacją.
// W tym kodzie konkretnie nie używasz funkcji z tego pliku, ale jest on dołączany.
include 'auth_utils.php';

// Zdefiniuj stałe lub zmienne przechowujące dane do połączenia z bazą danych.
$host = "localhost"; // Adres serwera bazy danych.
$uzytkownik_db = "root"; // Nazwa użytkownika bazy danych.
$haslo_db = ""; // Hasło użytkownika bazy danych (w tym przypadku puste).
$nazwa_bazy = "buty"; // Nazwa bazy danych, z którą chcemy się połączyć.

// Utwórz nowe połączenie z bazą danych MySQL przy użyciu klasy mysqli.
$polaczenie = new mysqli($host, $uzytkownik_db, $haslo_db, $nazwa_bazy);

// Sprawdź, czy wystąpił błąd podczas próby połączenia z bazą danych.
if ($polaczenie->connect_error) {
    // Jeśli połączenie nie powiodło się, wyświetl komunikat o błędzie i zakończ wykonywanie skryptu.
    die("Błąd połączenia z bazą danych: " . $polaczenie->connect_error);
}

// Sprawdź, czy żądanie do tego skryptu zostało wysłane metodą POST.
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    // Pobierz wartość 'id_produktu' z danych przesłanych metodą POST.
    $id_produktu = $_POST['id_produktu'];
    // Pobierz wartość 'ocena' z danych przesłanych metodą POST.
    $ocena = $_POST['ocena'];
    // Pobierz wartość 'imie' z danych POST i zabezpiecz ją przed atakami XSS za pomocą funkcji htmlspecialchars().
    $imie = htmlspecialchars($_POST['imie']);
    // Pobierz wartość 'opinia' z danych POST i zabezpiecz ją przed atakami XSS za pomocą funkcji htmlspecialchars().
    $opinia = htmlspecialchars($_POST['opinia']);

    // Przygotuj zapytanie SQL do wstawienia nowej opinii do tabeli 'opinie'.
    // Używamy prepared statements (parametryzowanych zapytań) dla bezpieczeństwa.
    $sql_wstaw_opinia = "INSERT INTO opinie (id_produktu, ocena, komentarz, imie) VALUES (?, ?, ?, ?)";
    // Przygotuj zapytanie SQL na utworzonym połączeniu z bazą danych.
    $stmt_wstaw_opinia = $polaczenie->prepare($sql_wstaw_opinia);
    // Powiąż parametry z zapytaniem SQL. "iiss" oznacza typy danych kolejnych parametrów:
    // i - integer (id_produktu, ocena)
    // s - string (komentarz, imie)
    $stmt_wstaw_opinia->bind_param("iiss", $id_produktu, $ocena, $opinia, $imie);

    // Wykonaj przygotowane zapytanie SQL.
    if ($stmt_wstaw_opinia->execute()) {
        // Jeśli zapytanie wykonało się pomyślnie, wyświetl komunikat o sukcesie.
        echo "Opinia dodana pomyślnie.";
    } else {
        // Jeśli wystąpił błąd podczas wykonywania zapytania, wyświetl komunikat o błędzie wraz z informacją o błędzie z bazy danych.
        echo "Błąd podczas dodawania opinii: " . $polaczenie->error;
    }
```

```php
    // Zamknij przygotowane zapytanie, aby zwolnić zasoby.
    $stmt_wstaw_opinia->close();
} else {
    // Jeśli żądanie nie było metodą POST, wyświetl komunikat o błędzie.
    echo "Błąd: Brak danych.";
}

// Zamknij połączenie z bazą danych, aby zwolnić zasoby.
$polaczenie->close();
?>
```

Kontakt

```php
<?php
session_start(); // Uruchomienie sesji
include 'auth_utils.php'; // Dołączenie pliku z funkcjami autoryzacji

// Sprawdzenie, czy użytkownik jest zalogowany i jaka jest jego rola
$zalogowany = isset($_SESSION['username']);
$rola = $_SESSION['rola'] ?? 'gość';  // Jeśli nie podano roli, przypisz 'gość'

// Obsługa formularza po jego wysłaniu (metoda POST)
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $con = mysqli_connect("localhost", "root", "", "buty"); // Połączenie z bazą danych

    // Sprawdzenie połączenia z bazą
    if (!$con) {
        $_SESSION['komunikat'] = "Błąd połączenia z bazą danych.";
        header("Location: " . $_SERVER['REQUEST_URI']);
        exit;
    }

    // Oczyszczenie danych z formularza przed zapisaniem do bazy
    $imie = mysqli_real_escape_string($con, $_POST["imie"]);
    $email = mysqli_real_escape_string($con, $_POST["email"]);
    $pytanie = mysqli_real_escape_string($con, $_POST["pytanie"]);

    // Zapytanie SQL - dodanie wiadomości do bazy danych
    $insert_query = "INSERT INTO wiadomosci (imie, email, pytanie)
                     VALUES ('$imie', '$email', '$pytanie')";

    // Sprawdzenie, czy zapytanie zostało wykonane
    if (mysqli_query($con, $insert_query)) {
        $_SESSION['komunikat'] = 'Wiadomość została wysłana! Dziękujemy za kontakt 😉';
    } else {
        $_SESSION['komunikat'] = 'Błąd podczas wysyłania wiadomości.';
    }

    mysqli_close($con); // Zamknięcie połączenia
    header("Location: " . $_SERVER['REQUEST_URI']); // Odświeżenie strony po wysłaniu
    exit;
}
?>
```

- funkcje JavaScript odpowiedzialne za interaktywność menu i slidera

```
<script>
    var indeks = 1; // Inicjalizacja indeksu pierwszego slajdu.
    pok(indeks); // Wyświetlenie pierwszego slajdu po załadowaniu strony.

    // Funkcja do zmiany slajdu (n - o ile slajdów przesunąć).
    function plus(n) {
        pok(indeks += n); // Zwiększenie/zmniejszenie indeksu i wyświetlenie slajdu.
    }

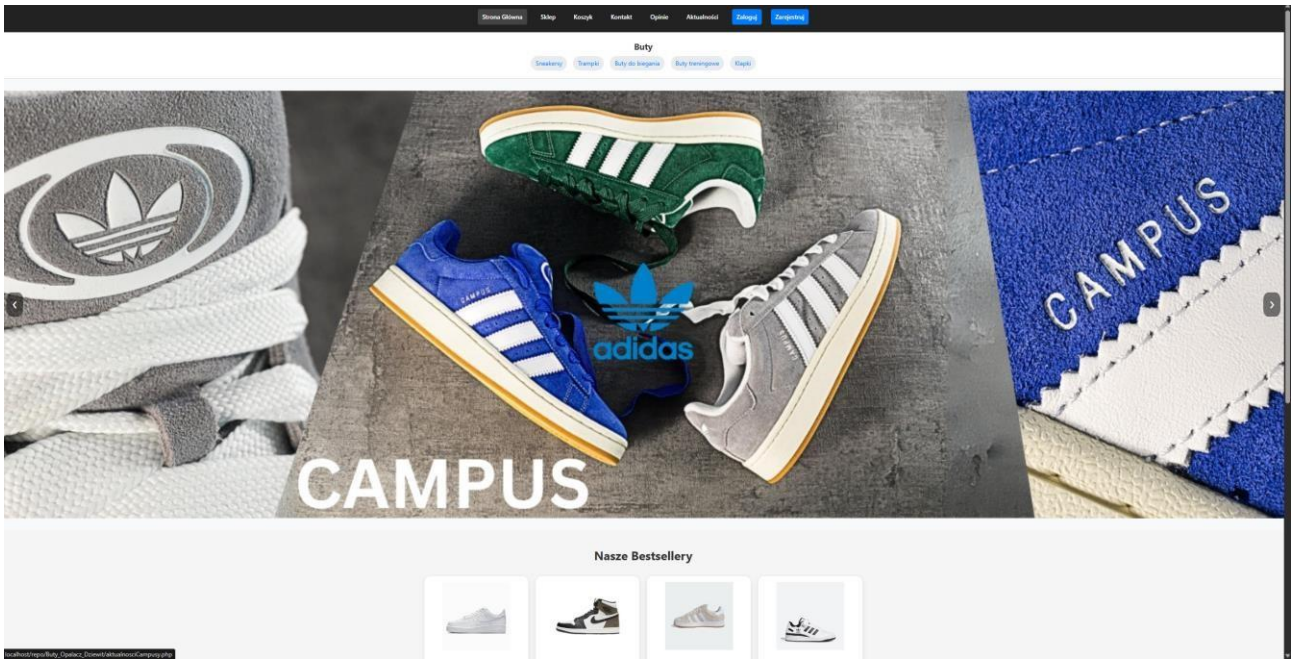
    // Funkcja wyświetlająca slajd o podanym indeksie (n).
    function pok(n) {
        var i;
        var x = document.getElementsByClassName("pokazslajdow"); // Pobranie wszystkich elementów z klasą "pokazslajdow".
        // Jeśli indeks jest większy niż liczba slajdów, wróć do pierwszego.
        if (n > x.length) {
            indeks = 1
        }
        // Jeśli indeks jest mniejszy niż 1, przejdź do ostatniego slajdu.
        if (n < 1) {
            indeks = x.length
        }
        // Ukrycie wszystkich slajdów.
        for (i = 0; i < x.length; i++) {
            x[i].style.display = "none";
        }
        // Wyświetlenie slajdu o aktualnym indeksie.
        x[indeks - 1].style.display = "block";
    }
</script>
<div class="pokaz">
    <button onclick="plus(-1)">&#10094;</button>

    <a href="aktualnosciCampusy.php"><img class="pokazslajdow" src="img/slider/elo.jpg" style="width:100%"></a>
    <a href="aktualnosciJordany.php"><img class="pokazslajdow" src="img/slider/jord.jpg" style="width:100%"></a>
    <a href="aktualnosciKlapki.php"><img class="pokazslajdow" src="img/slider/klapki.jpg" style="width:100%"></a>

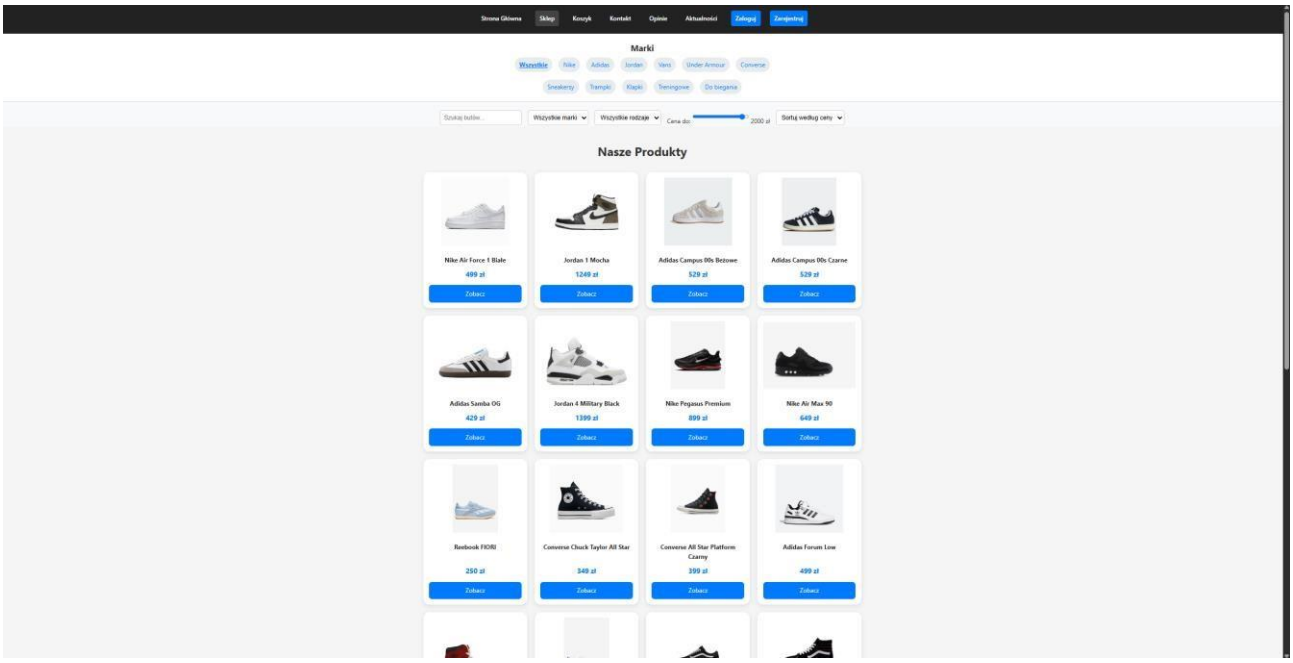
    <button onclick="plus(1)">&#10095;</button>
</div>
```

10.2. Zrzuty ekranu działania systemu

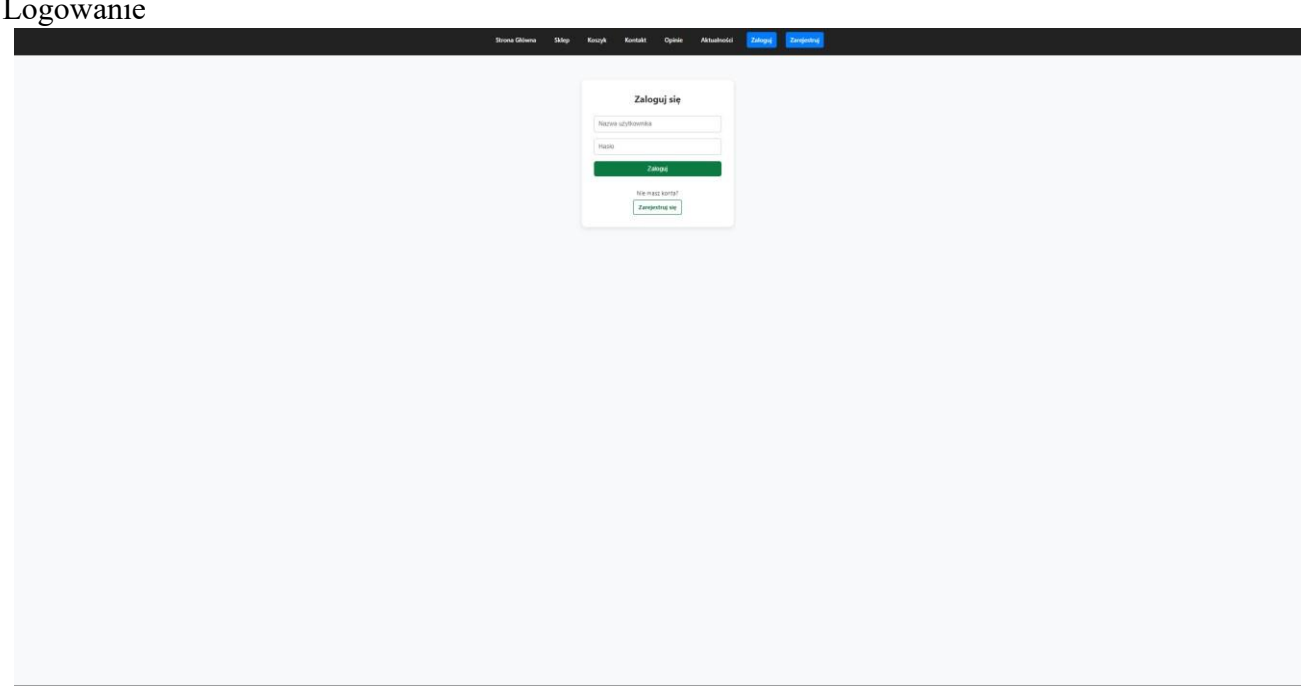
Strona Główna



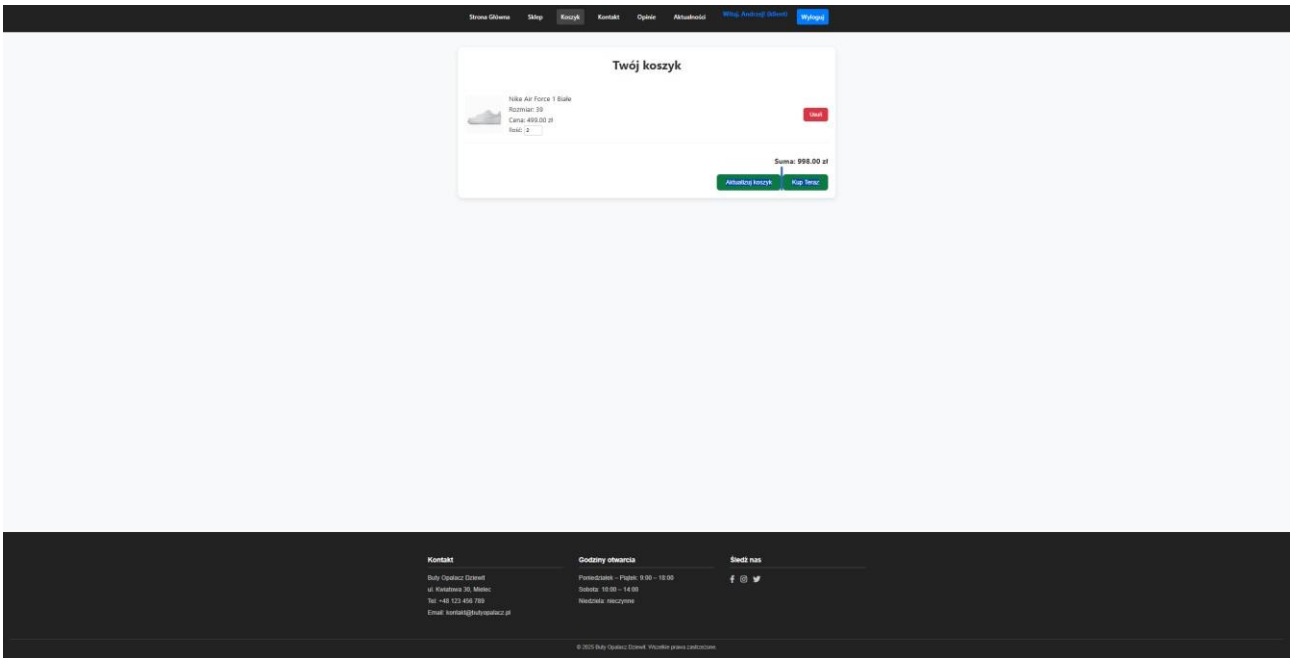
Sklep



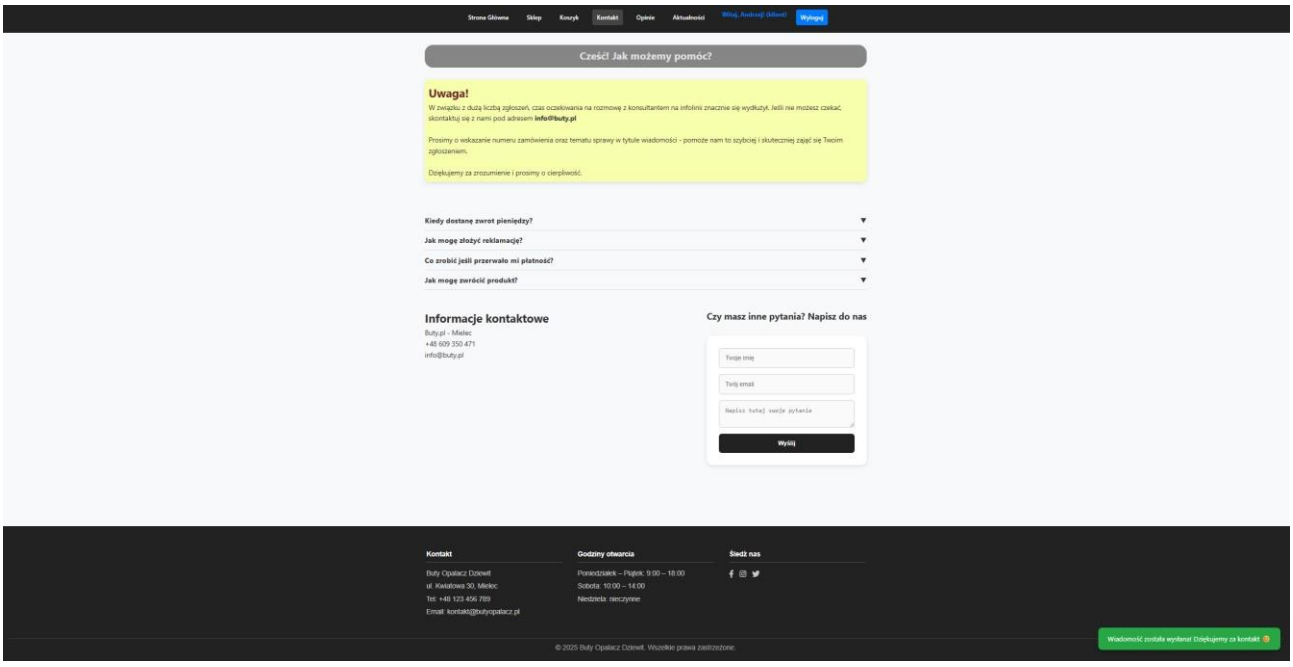
Logowanie



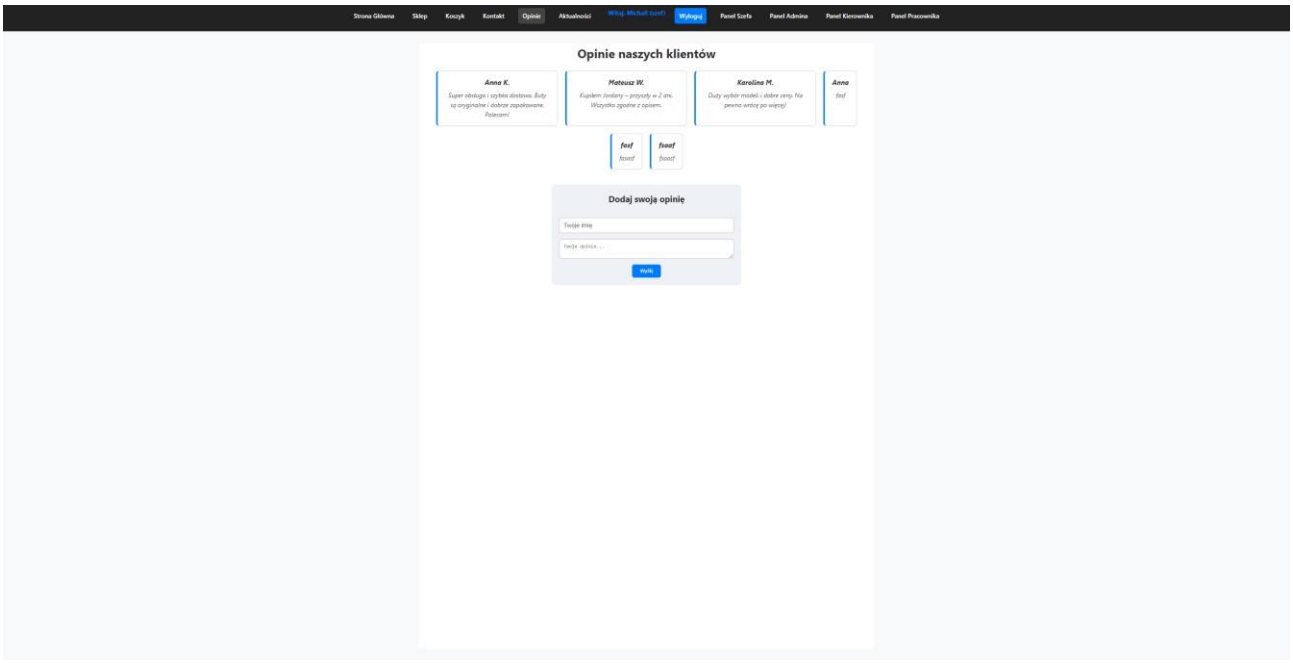
Koszyk



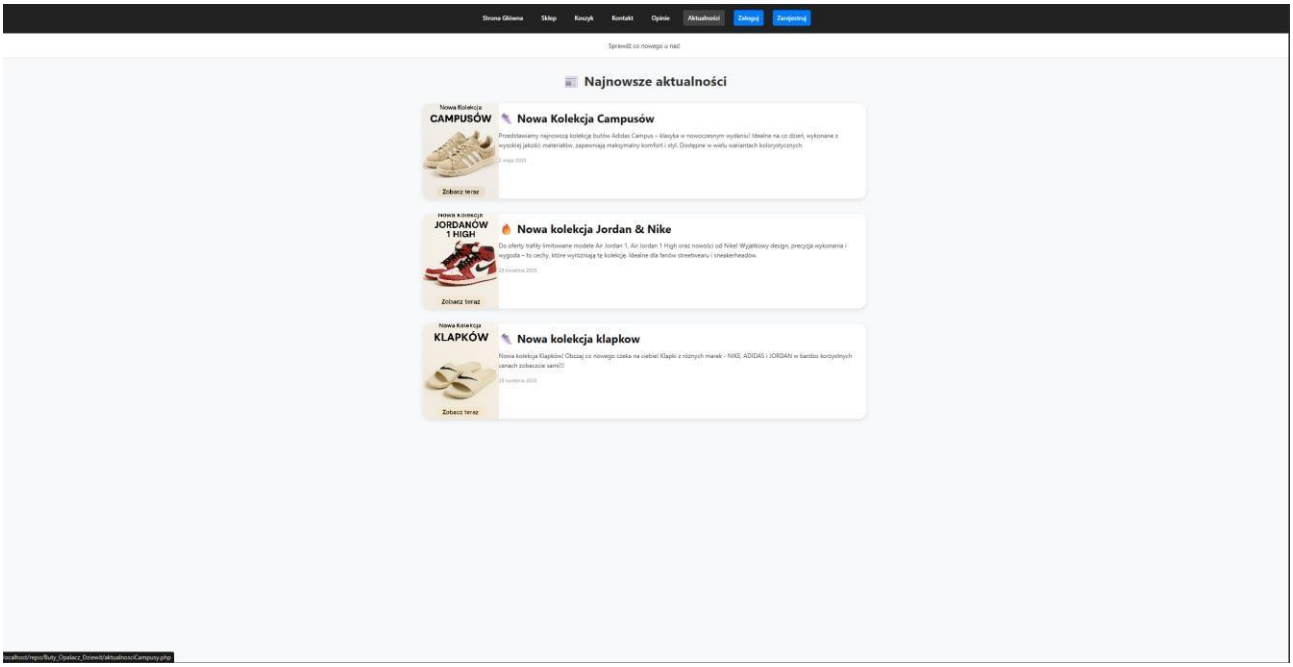
Kontakt Po wyslaniu pytania



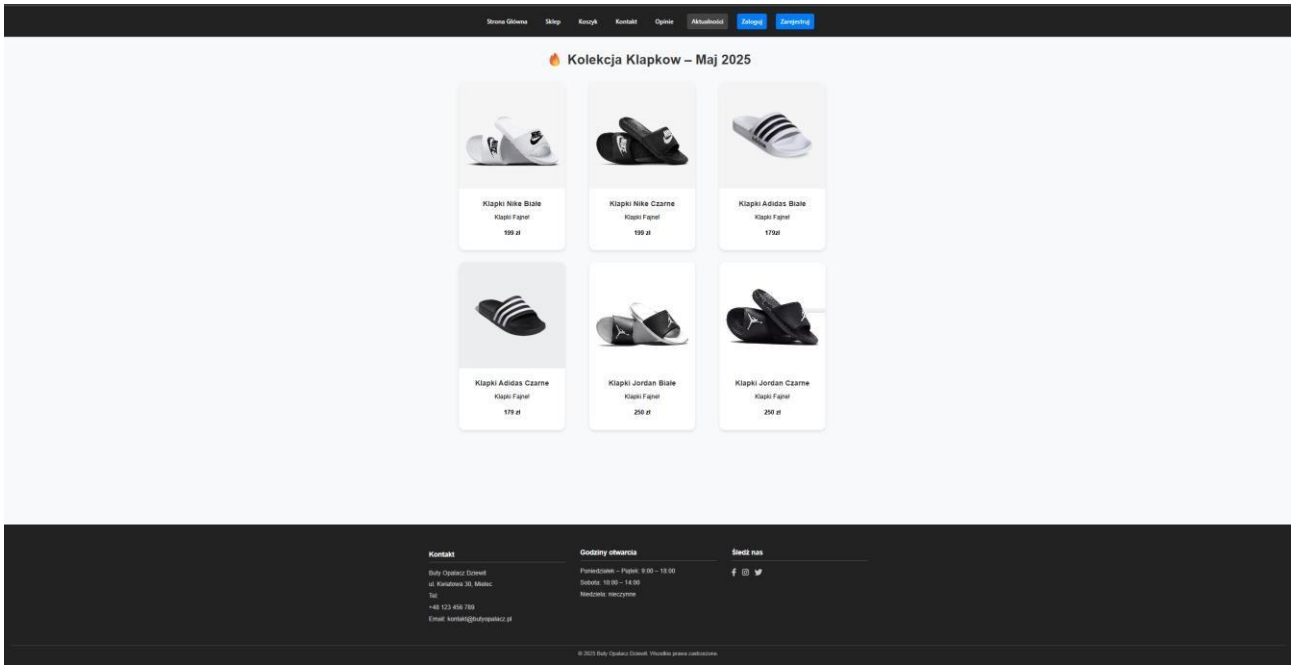
Opinie



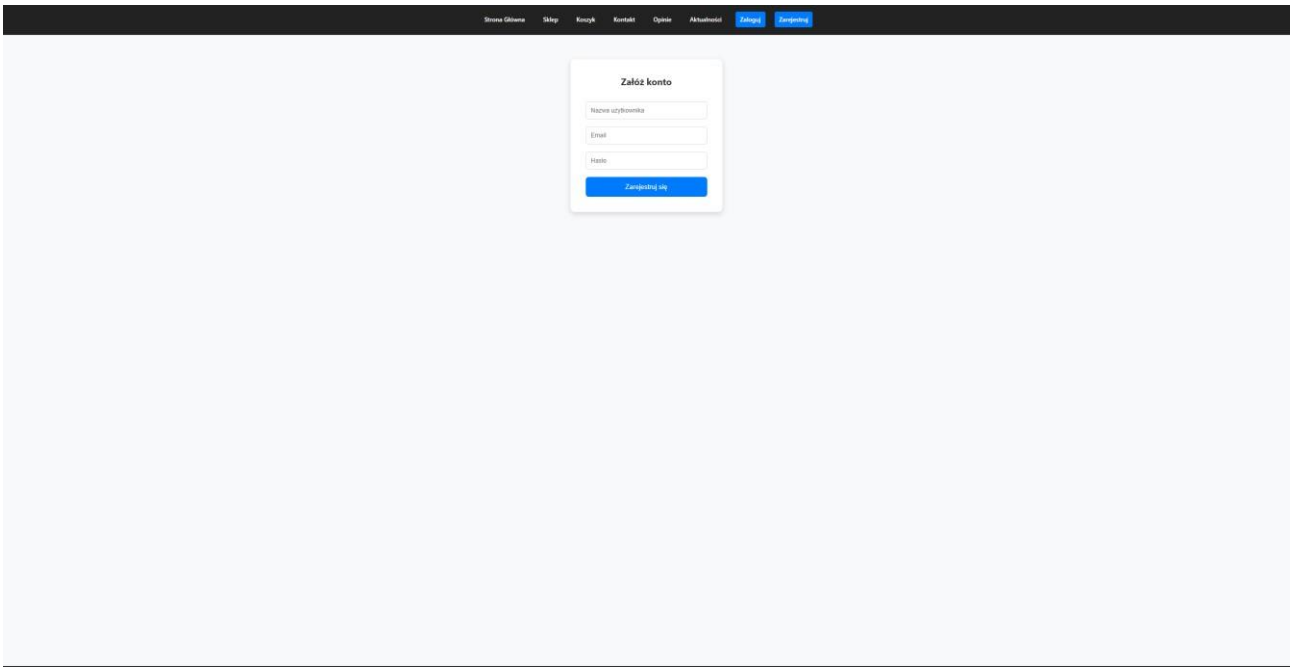
Aktualnosci



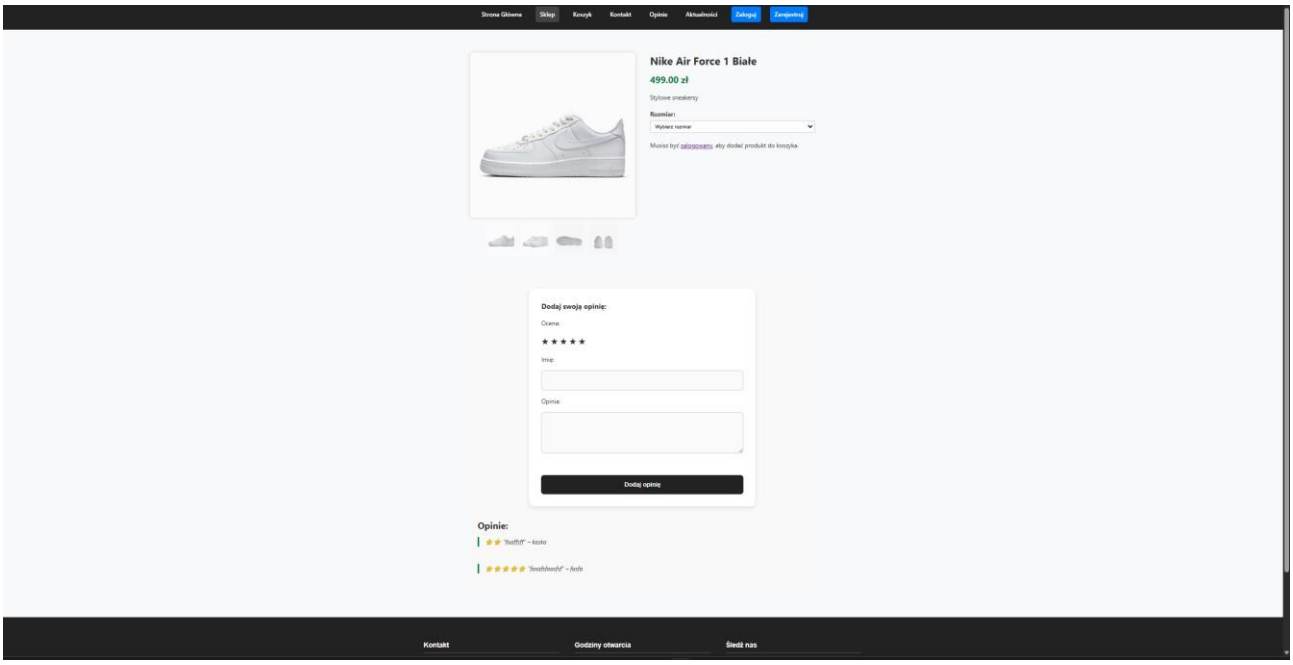
Aktualnosci np klapkow



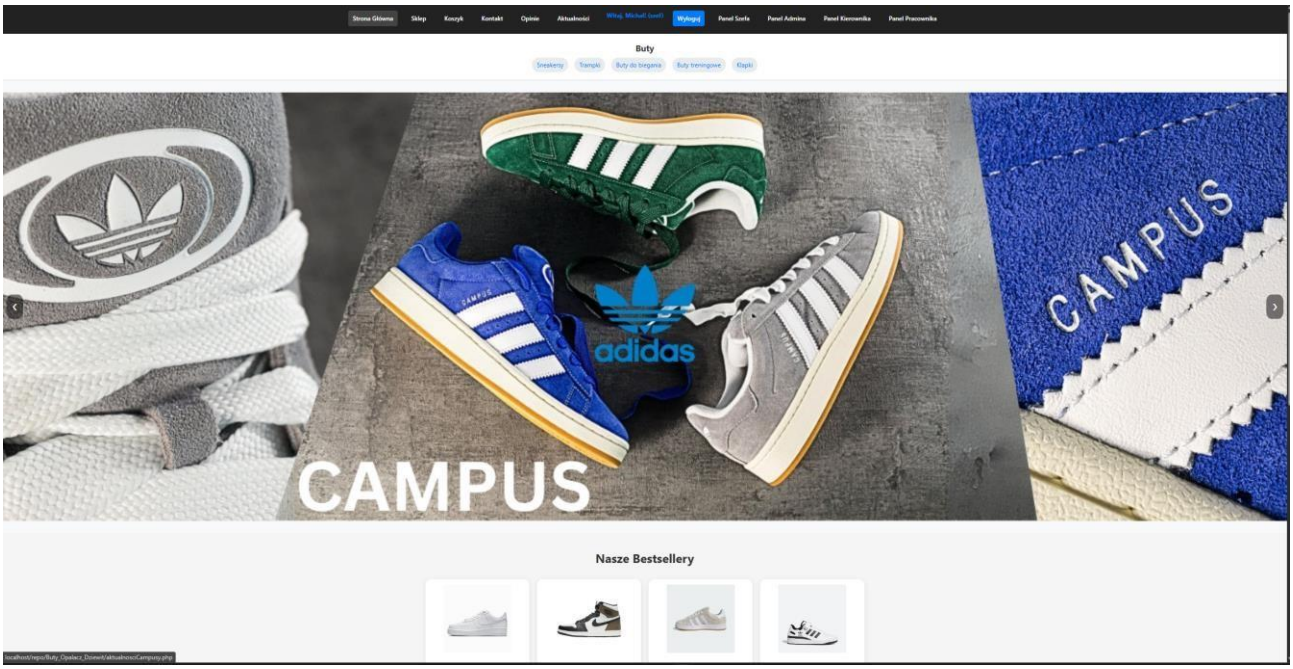
Rejestracja



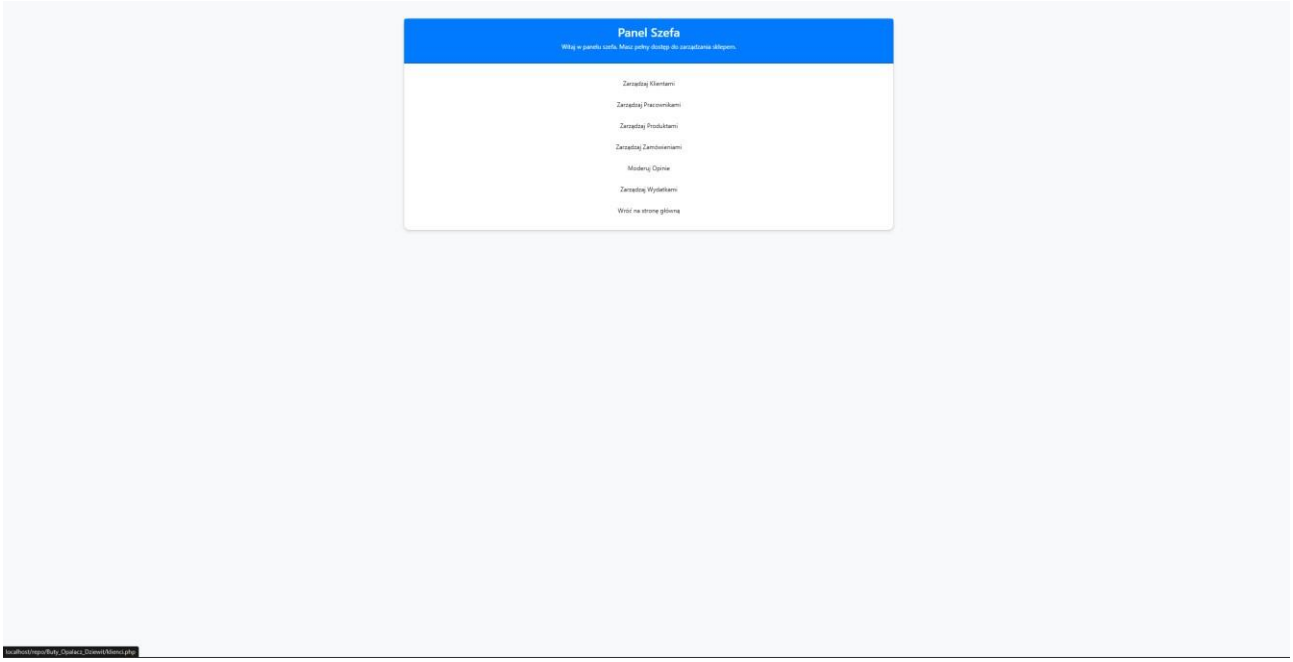
Strona Produktu



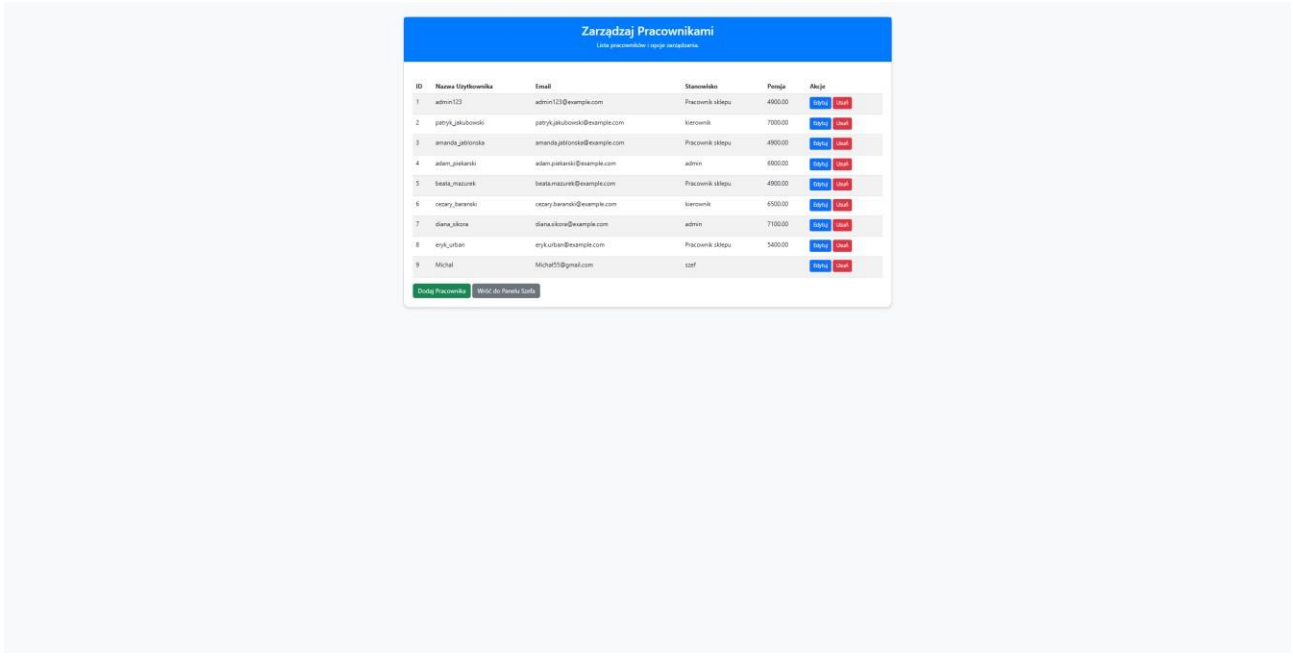
Zalogowanie jako szef masz dostep do wszystkich paneli



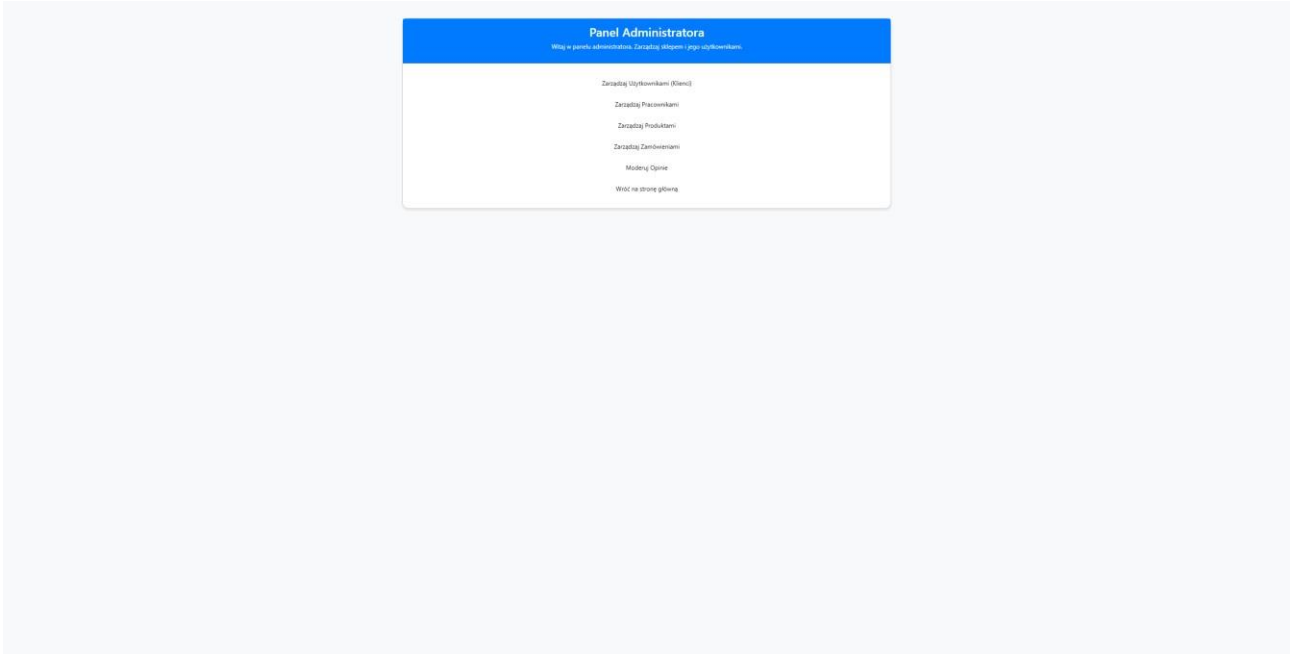
Panel szefa



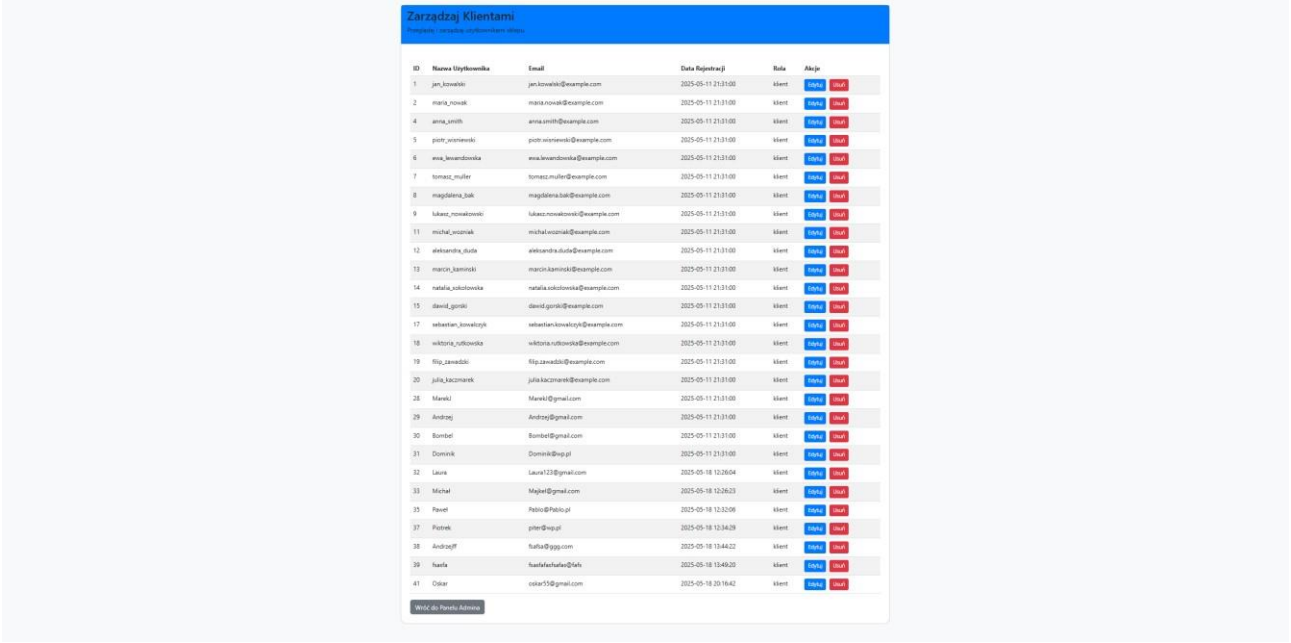
Tutaj np Pracownicy mozna usuwac edytowac dodawac



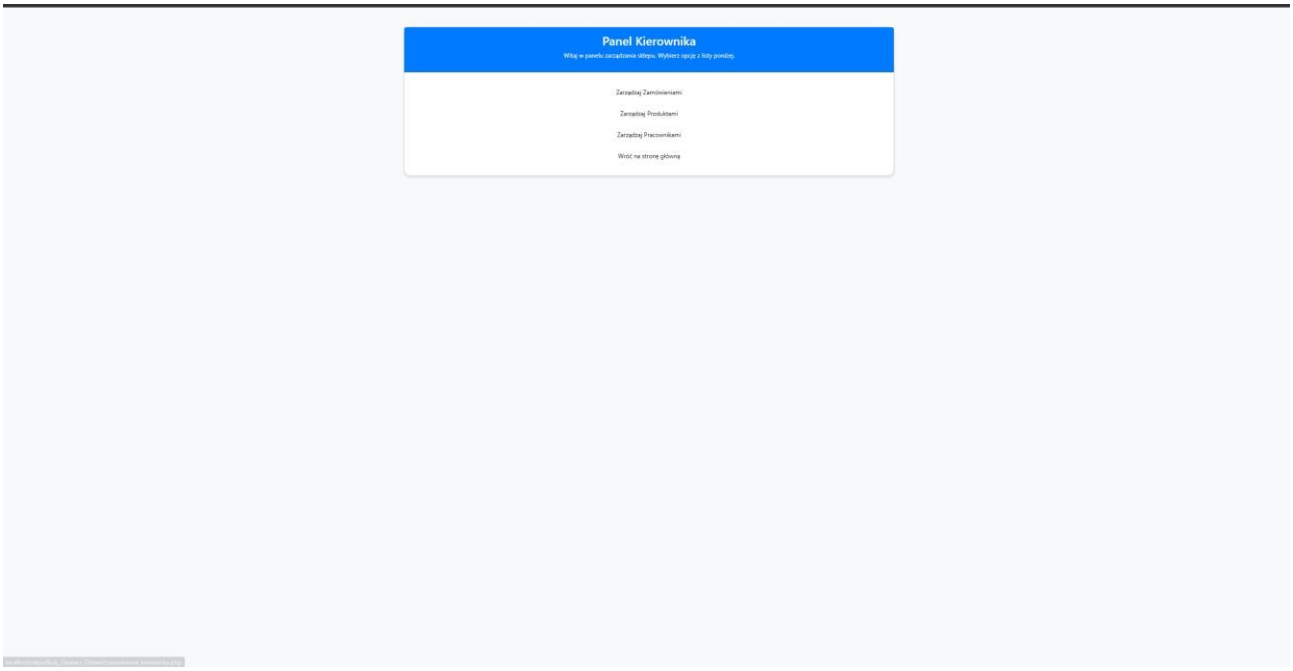
Panel Admina



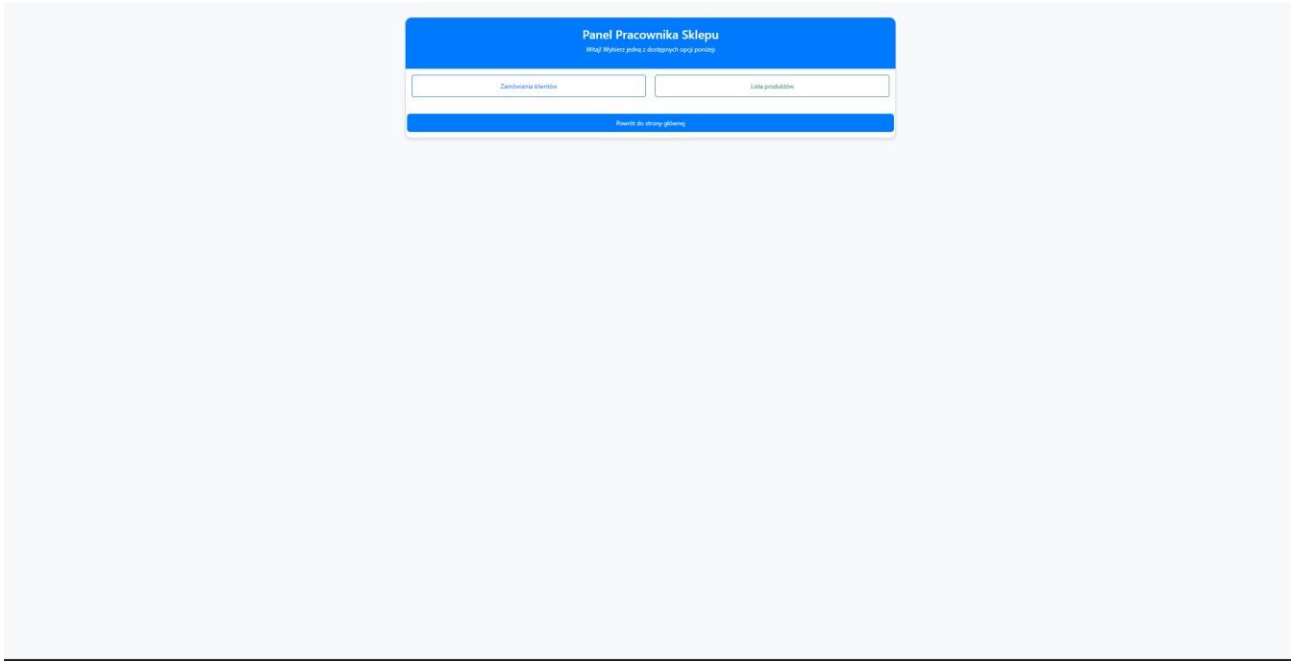
Np zarzadzanie klientami



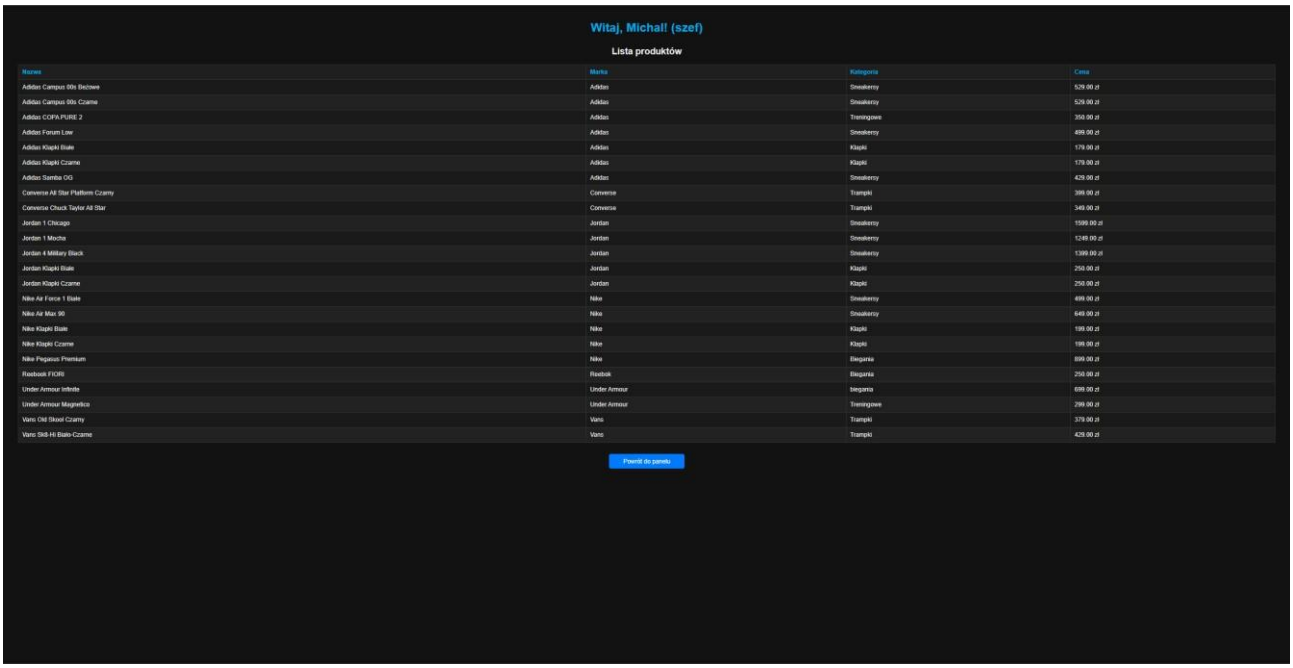
Panel kierownika



Panel Pracownika Sklepu



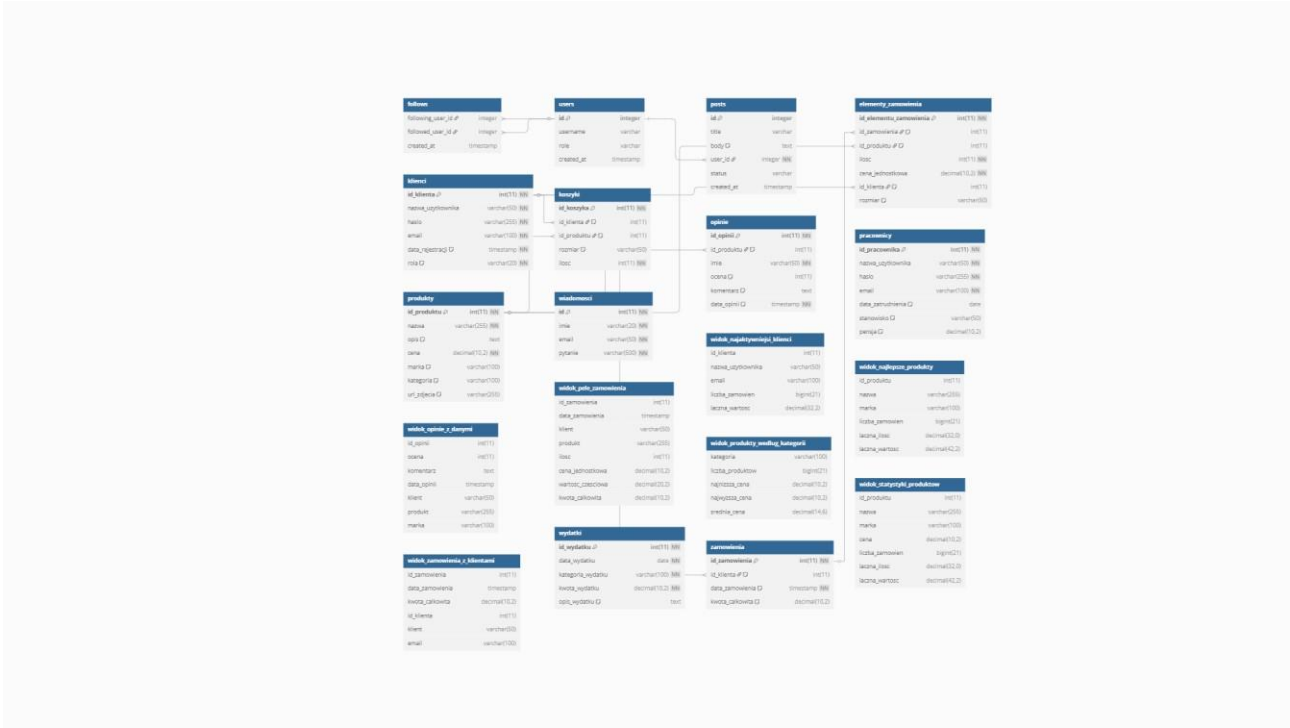
Np produkty



10.3. Diagram bazy danych

Dołączony został diagram ER przedstawiający:

- strukturę bazy danych,
- powiązania między tabelami,
- klucze główne i obce.



10.4. Lista błędów i pomysłów na ulepszenia

Zidentyfikowane błędy i problemy:

- **Brak walidacji unikalności adresu e-mail podczas rejestracji** – obecnie system umożliwia utworzenie wielu kont z tym samym adresem e-mail, co może prowadzić do problemów z logowaniem i przypomnieniem hasła.
- **Niewystarczająco precyzyjne komunikaty błędów przy logowaniu** – w przypadku błędnych danych użytkownik nie otrzymuje jasnej informacji, czy problem dotyczy loginu, hasła czy braku konta.

Propozycje usprawnień i dalszego rozwoju:

- **Integracja z systemami płatności online** – dodanie obsługi popularnych metod płatności (np. Przelewy24, PayU, BLIK) znacząco zwiększy użyteczność systemu i ułatwi finalizację zakupów.
- **Automatyczne powiadomienia e-mail** – wdrożenie wysyłki wiadomości e-mail przy rejestracji, złożeniu zamówienia, zmianie statusu zamówienia itp. zwiększy transparentność i zaufanie użytkowników.
- **Zaawansowana analityka użytkowników i sprzedaży** – integracja z narzędziami typu Google Analytics lub Matomo umożliwi gromadzenie danych o zachowaniach użytkowników, źródłach ruchu i efektywności działań marketingowych.

- **Lepsza obsługa błędów i walidacji formularzy** – ujednolicenie i rozszerzenie komunikatów błędów oraz wdrożenie walidacji dynamicznej (np. AJAX) poprawi doświadczenie użytkownika.

- **Rozszerzenie panelu administracyjnego** – umożliwienie filtrowania zamówień, zarządzania użytkownikami i generowania raportów ułatwi zarządzanie sklepem.